

Te preparo el paquete técnico inicial de implementación para Debacu con:

1. estructura de carpetas
2. tipos TypeScript
3. conector base abstracto
4. TesiproConnector placeholder
5. repositorios
6. servicio de sincronización
7. mapper base
8. ejemplo de Edge Function de sync
9. orden de implementación realista
10. riesgos y decisiones

Voy a plantearlo para tu stack real: **Supabase Edge Functions + TypeScript + Postgres/Supabase.**

1. Objetivo del paquete técnico inicial

Este paquete no intenta resolver ya toda la integración PMS.

Su objetivo es dejar una base limpia para que luego puedas conectar:

- Tesipro / Ulyses
- Cloudbeds
- Mews
- apaleo
- Opera

sin rehacer arquitectura.

La idea es separar claramente:

- **cliente PMS**
 - **normalización**
 - **persistencia**
 - **orquestración de sync**
 - **Edge Function de entrada**
-

2. Estructura de carpetas recomendada

Yo lo montaría así dentro de supabase/functions/:

```
supabase/  
  functions/  
    _shared/  
      cors.ts  
      auth.ts  
      env.ts  
      errors.ts  
      db.ts  
      logger.ts  
    pms_sync_run/  
      index.ts  
    _shared/pms/  
      types/  
        pms.types.ts  
        pms-canonical.types.ts  
        tesipro.types.ts  
      connectors/  
        PMSBaseConnector.ts  
        TesiproConnector.ts  
      mappers/  
        reservationStatus.mapper.ts  
        stayStatus.mapper.ts  
        roomStatus.mapper.ts  
        tesipro.mapper.ts  
      repositories/  
        PmsConnectionRepository.ts  
        PmsSyncJobRepository.ts  
        PmsRawRecordRepository.ts  
        PmsRoomTypeRepository.ts  
        PmsRoomRepository.ts  
        PmsGuestRepository.ts  
        PmsReservationRepository.ts  
        PmsStayRepository.ts  
      services/  
        PmsSyncService.ts  
        PmsNormalizationService.ts  
        PmsProviderFactory.ts  
      utils/  
        hash.ts  
        dates.ts  
        chunk.ts
```

3. Responsabilidad de cada capa

`_shared/pms/types`

Tipos internos del sistema:

- interfaces canónicas
- tipos raw
- contratos de conectores

connectors

Cliente/adaptador por PMS:

- autenticación
- fetch al PMS
- paginación
- lectura de entidades

mappers

Transformación de raw PMS a modelo canónico Debacu.

repositories

Acceso a tablas Postgres/Supabase.

services

Orquestación de procesos:

- elegir provider
- lanzar jobs
- normalizar
- persistir
- manejar errores

pms_sync_run/index.ts

Punto de entrada Edge Function.

4. Tipos base TypeScript

4.1 pms.types.ts

```
// supabase/functions/_shared/pms/types/pms.types.ts
export type PmsProviderCode =
  | "TESIPRO_ULYSES"
  | "CLOUDBEDS"
  | "MEWS"
  | "APALEO"
  | "OPERA";
export type PmsEntityType =
  | "PROPERTY"
  | "ROOM_TYPE"
```

```

    | "ROOM"
    | "GUEST"
    | "RESERVATION"
    | "STAY"
    | "FOLIO";
export type PmsSyncMode = "FULL" | "INCREMENTAL";
export type PmsConnectionStatus =
    | "PENDING"
    | "ACTIVE"
    | "ERROR"
    | "DISCONNECTED";
export interface PmsConnection {
    id: string;
    org_id: string;
    property_id: string | null;
    provider_code: PmsProviderCode;
    environment: "sandbox" | "production";
    external_account_id: string | null;
    external_group_id: string | null;
    auth_mode: string;
    status: PmsConnectionStatus;
    last_sync_at: string | null;
    last_success_sync_at: string | null;
    last_error_at: string | null;
    last_error_message: string | null;
    created_at: string;
    updated_at: string;
}
export interface PmsSyncRunInput {
    connectionId: string;
    entityType: PmsEntityType;
    syncMode: PmsSyncMode;
    propertyId?: string | null;
    cursor?: string | null;
    modifiedSince?: string | null;
    limit?: number;
}
export interface PmsSyncContext {
    connection: PmsConnection;
    entityType: PmsEntityType;
    syncMode: PmsSyncMode;
    propertyId: string | null;
    cursor?: string | null;
    modifiedSince?: string | null;
    limit?: number;
    syncJobId: string;
}
export interface PmsPagedResult<T> {
    items: T[];
    nextCursor: string | null;
    sourceUpdatedAtMax?: string | null;
}
export interface PmsRawEnvelope<T = unknown> {
    providerCode: PmsProviderCode;
    entityType: PmsEntityType;
    externalId: string;
    externalParentId?: string | null;
    externalUpdatedAt?: string | null;
    payload: T;
}

```

4.2 pms-canonical.types.ts

```
// supabase/functions/_shared/pms/types/pms-canonical.types.ts
export type CanonicalReservationStatus =
  | "PENDING"
  | "CONFIRMED"
  | "CHECKED_IN"
  | "CHECKED_OUT"
  | "CANCELLED"
  | "NO_SHOW"
  | "IN_HOUSE"
  | "UNKNOWN";
export type CanonicalStayStatus =
  | "EXPECTED"
  | "IN_HOUSE"
  | "CHECKED_OUT"
  | "CANCELLED"
  | "NO_SHOW"
  | "UNKNOWN";
export type CanonicalRoomOperationalStatus =
  | "AVAILABLE"
  | "OUT_OF_SERVICE"
  | "DIRTY"
  | "CLEAN"
  | "UNKNOWN";
export interface CanonicalRoomType {
  org_id: string;
  property_id: string;
  connection_id: string;
  external_room_type_id: string;
  code: string | null;
  name: string;
  description: string | null;
  capacity_adults: number | null;
  capacity_children: number | null;
  max_occupancy: number | null;
  is_active: boolean;
  raw_status: string | null;
  source_updated_at: string | null;
}
export interface CanonicalRoom {
  org_id: string;
  property_id: string;
  connection_id: string;
  external_room_id: string;
  external_room_type_id?: string | null;
  room_number: string | null;
  room_name: string | null;
  floor: string | null;
  building: string | null;
  is_active: boolean;
  operational_status: CanonicalRoomOperationalStatus;
  raw_status: string | null;
  source_updated_at: string | null;
}
export interface CanonicalGuest {
  org_id: string;
  property_id: string | null;
  connection_id: string;
  external_guest_id: string;
  external_profile_id: string | null;
  first_name: string | null;
  last_name: string | null;
}
```

```

    full_name: string | null;
    email: string | null;
    phone: string | null;
    document_type: string | null;
    document_number: string | null;
    country_code: string | null;
    nationality_code: string | null;
    birth_date: string | null;
    gender: string | null;
    is_company: boolean | null;
    company_name: string | null;
    is_active: boolean;
    source_updated_at: string | null;
}
export interface CanonicalReservation {
    org_id: string;
    property_id: string;
    connection_id: string;
    external_reservation_id: string;
    external_confirmation_code: string | null;
    external_group_reservation_id: string | null;
    external_guest_id?: string | null;
    external_primary_room_type_id?: string | null;
    external_assigned_room_id?: string | null;
    status: CanonicalReservationStatus;
    raw_status: string | null;
    booked_at: string | null;
    check_in_date: string | null;
    check_out_date: string | null;
    nights: number | null;
    adults: number | null;
    children: number | null;
    infants: number | null;
    rooms_count: number | null;
    channel_code: string | null;
    channel_name: string | null;
    segment_code: string | null;
    segment_name: string | null;
    rate_plan_code: string | null;
    rate_plan_name: string | null;
    currency_code: string | null;
    room_revenue_amount: number | null;
    total_amount: number | null;
    paid_amount: number | null;
    balance_amount: number | null;
    cancellation_at: string | null;
    source_updated_at: string | null;
}
export interface CanonicalStay {
    org_id: string;
    property_id: string;
    connection_id: string;
    external_stay_id: string | null;
    external_reservation_id?: string | null;
    external_guest_id?: string | null;
    external_room_id?: string | null;
    external_room_type_id?: string | null;
    stay_status: CanonicalStayStatus;
    raw_status: string | null;
    arrival_scheduled_at: string | null;
    departure_scheduled_at: string | null;
    check_in_at: string | null;
    check_out_at: string | null;
    adults: number | null;

```

```
children: number | null;
currency_code: string | null;
room_revenue_amount: number | null;
total_amount: number | null;
source_updated_at: string | null;
}
```

4.3 tesipro.types.ts

Esto es provisional, porque no tenemos la API real cerrada.

```
// supabase/functions/_shared/pms/types/tesipro.types.ts
export interface TesiproReservationRaw {
  booking_id?: string;
  reservation_id?: string;
  booking_status?: string;
  status?: string;
  client_name?: string;
  guest_name?: string;
  arrival_date?: string;
  departure_date?: string;
  created_at?: string;
  updated_at?: string;
  email?: string;
  phone?: string;
  room_type_id?: string;
  room_id?: string;
  adults?: number | string;
  children?: number | string;
  total_amount?: number | string;
  currency?: string;
  channel?: string;
  segment?: string;
  rate_plan?: string;
}
export interface TesiproRoomTypeRaw {
  id?: string;
  code?: string;
  name?: string;
  description?: string;
  capacity?: number | string;
  max_occupancy?: number | string;
  active?: boolean | string;
  updated_at?: string;
}
export interface TesiproRoomRaw {
  id?: string;
  room_id?: string;
  room_number?: string;
  room_name?: string;
  room_type_id?: string;
  floor?: string;
  building?: string;
  status?: string;
  active?: boolean | string;
  updated_at?: string;
}
```

5. Utilidades comunes

hash.ts

```
// supabase/functions/_shared/pms/utils/hash.ts
export async function sha256Hex(input: unknown): Promise<string> {
  const text = typeof input === "string" ? input : JSON.stringify(input);
  const data = new TextEncoder().encode(text);
  const hashBuffer = await crypto.subtle.digest("SHA-256", data);
  return Array.from(new Uint8Array(hashBuffer))
    .map((b) => b.toString(16).padStart(2, "0"))
    .join("");
}
```

chunk.ts

```
// supabase/functions/_shared/pms/utils/chunk.ts
export function chunkArray<T>(items: T[], size = 100): T[][] {
  if (size <= 0) throw new Error("chunk size must be > 0");
  const chunks: T[][] = [];
  for (let i = 0; i < items.length; i += size) {
    chunks.push(items.slice(i, i + size));
  }
  return chunks;
}
```

6. Conector base abstracto

PMSBaseConnector.ts

```
// supabase/functions/_shared/pms/connectors/PMSBaseConnector.ts
import {
  PmsEntityType,
  PmsPagedResult,
  PmsSyncContext,
} from "../../types/pms.types.ts";
export abstract class PMSBaseConnector {
  abstract providerCode: string;
  abstract fetchRoomTypes(
    ctx: PmsSyncContext,
  ): Promise<PmsPagedResult<Record<string, unknown>>>>;
  abstract fetchRooms(
    ctx: PmsSyncContext,
  ): Promise<PmsPagedResult<Record<string, unknown>>>>;
  abstract fetchGuests(
    ctx: PmsSyncContext,
  ): Promise<PmsPagedResult<Record<string, unknown>>>>;
  abstract fetchReservations(
    ctx: PmsSyncContext,
  ): Promise<PmsPagedResult<Record<string, unknown>>>>;
  abstract fetchStays(
    ctx: PmsSyncContext,
  ): Promise<PmsPagedResult<Record<string, unknown>>>>;
  async fetchByEntity(
    ctx: PmsSyncContext,
```



```

): Promise<PmsPagedResult<Record<string, unknown>>> {
  switch (ctx.entityType as PmsEntityType) {
    case "ROOM_TYPE":
      return await this.fetchRoomTypes(ctx);
    case "ROOM":
      return await this.fetchRooms(ctx);
    case "GUEST":
      return await this.fetchGuests(ctx);
    case "RESERVATION":
      return await this.fetchReservations(ctx);
    case "STAY":
      return await this.fetchStays(ctx);
    default:
      throw new Error(`Unsupported entity type for connector: ${ctx.entityType}`);
  }
}
}

```

7. Tesipro connector placeholder

No voy a inventarme endpoints reales. Lo dejo preparado para completar.

TesiproConnector.ts

```

// supabase/functions/_shared/pms/connectors/TesiproConnector.ts
import { PMSBaseConnector } from "../PMSBaseConnector.ts";
import { PmsPagedResult, PmsSyncContext } from "../../types/pms.types.ts";
export class TesiproConnector extends PMSBaseConnector {
  providerCode = "TESIPRO_ULYSES";
  private getBaseUrl(ctx: PmsSyncContext): string {
    const envUrl = Deno.env.get("TESIPRO_API_BASE_URL");
    if (!envUrl) {
      throw new Error("TESIPRO_API_BASE_URL is not configured");
    }
    return envUrl;
  }
  private async requestJson(
    _ctx: PmsSyncContext,
    _path: string,
    _query?: Record<string, string | number | boolean | null | undefined>,
  ): Promise<unknown> {
    throw new Error(
      "TesiproConnector.requestJson not implemented yet. Pending API documentation and real auth model.",
    );
  }
  async fetchRoomTypes(
    ctx: PmsSyncContext,
  ): Promise<PmsPagedResult<Record<string, unknown>>> {
    this.getBaseUrl(ctx);
    throw new Error("Tesipro fetchRoomTypes pending API validation");
  }
  async fetchRooms(
    ctx: PmsSyncContext,
  ): Promise<PmsPagedResult<Record<string, unknown>>> {
    this.getBaseUrl(ctx);
    throw new Error("Tesipro fetchRooms pending API validation");
  }
  async fetchGuests(

```

```

    ctx: PmsSyncContext,
  ): Promise<PmsPagedResult<Record<string, unknown>>> {
    this.getBaseUrl(ctx);
    throw new Error("Tesipro fetchGuests pending API validation");
  }
  async fetchReservations(
    ctx: PmsSyncContext,
  ): Promise<PmsPagedResult<Record<string, unknown>>> {
    this.getBaseUrl(ctx);
    throw new Error("Tesipro fetchReservations pending API validation");
  }
  async fetchStays(
    ctx: PmsSyncContext,
  ): Promise<PmsPagedResult<Record<string, unknown>>> {
    this.getBaseUrl(ctx);
    throw new Error("Tesipro fetchStays pending API validation");
  }
}

```

Esto está hecho a propósito. Mejor dejarlo limpio y explícito que fingir un conector que luego no sirve.

8. Factory de proveedores

PmsProviderFactory.ts

```

// supabase/functions/_shared/pms/services/PmsProviderFactory.ts
import { PMSBaseConnector } from "../../connectors/PMSBaseConnector.ts";
import { TesiproConnector } from "../../connectors/TesiproConnector.ts";
import { PmsProviderCode } from "../../types/pms.types.ts";
export class PmsProviderFactory {
  static create(providerCode: PmsProviderCode): PMSBaseConnector {
    switch (providerCode) {
      case "TESIPRO_ULYSES":
        return new TesiproConnector();
      default:
        throw new Error(`Provider not implemented: ${providerCode}`);
    }
  }
}

```

9. Mapper base de estados

reservationStatus.mapper.ts

```

// supabase/functions/_shared/pms/mappers/reservationStatus.mapper.ts
import { CanonicalReservationStatus } from "../../types/pms-canonical.types.ts";
export function mapReservationStatus(rawValue?: string | null):
CanonicalReservationStatus {
  const normalized = (rawValue ?? "").trim().toUpperCase();
  switch (normalized) {
    case "PENDING":
    case "ON_HOLD":
      return "PENDING";
    case "CONFIRMED":

```

```

    case "BOOKED":
    case "RESERVED":
        return "CONFIRMED";
    case "CHECKED_IN":
        return "CHECKED_IN";
    case "CHECKED_OUT":
        return "CHECKED_OUT";
    case "INHOUSE":
    case "IN_HOUSE":
    case "STARTED":
        return "IN_HOUSE";
    case "CANCELLED":
    case "CANCELED":
        return "CANCELLED";
    case "NO_SHOW":
    case "NOSHOW":
        return "NO_SHOW";
    default:
        return "UNKNOWN";
}
}

```

stayStatus.mapper.ts

```

// supabase/functions/_shared/pms/mappers/stayStatus.mapper.ts
import { CanonicalStayStatus } from "../types/pms-canonical.types.ts";
export function mapStayStatus(rawValue?: string | null): CanonicalStayStatus {
    const normalized = (rawValue ?? "").trim().toUpperCase();
    switch (normalized) {
        case "EXPECTED":
        case "DUE_IN":
            return "EXPECTED";
        case "INHOUSE":
        case "IN_HOUSE":
        case "STARTED":
            return "IN_HOUSE";
        case "CHECKED_OUT":
        case "DUE_OUT":
            return "CHECKED_OUT";
        case "CANCELLED":
        case "CANCELED":
            return "CANCELLED";
        case "NO_SHOW":
        case "NOSHOW":
            return "NO_SHOW";
        default:
            return "UNKNOWN";
    }
}

```

10. Mapper Tesipro provisional

tesipro.mapper.ts

```

// supabase/functions/_shared/pms/mappers/tesipro.mapper.ts
import {
    CanonicalReservation,

```

```

    CanonicalRoom,
    CanonicalRoomType,
} from "../types/pms-canonical.types.ts";
import {
    TesiproReservationRaw,
    TesiproRoomRaw,
    TesiproRoomTypeRaw,
} from "../types/tesipro.types.ts";
import { mapReservationStatus } from "../reservationStatus.mapper.ts";
function toNullableString(v: unknown): string | null {
    if (v === null || v === undefined) return null;
    const s = String(v).trim();
    return s.length ? s : null;
}
function toNullableNumber(v: unknown): number | null {
    if (v === null || v === undefined || v === "") return null;
    const n = Number(v);
    return Number.isFinite(n) ? n : null;
}
function toNullableBoolean(v: unknown): boolean | null {
    if (v === null || v === undefined) return null;
    if (typeof v === "boolean") return v;
    const s = String(v).trim().toLowerCase();
    if (["true", "1", "yes", "y", "si"].includes(s)) return true;
    if (["false", "0", "no", "n"].includes(s)) return false;
    return null;
}
export function mapTesiproRoomType(
    raw: TesiproRoomTypeRaw,
    base: { orgId: string; propertyId: string; connectionId: string },
): CanonicalRoomType {
    const cap = toNullableNumber(raw.capacity);
    const maxOcc = toNullableNumber(raw.max_occupancy);
    return {
        org_id: base.orgId,
        property_id: base.propertyId,
        connection_id: base.connectionId,
        external_room_type_id: toNullableString(raw.id) ?? "",
        code: toNullableString(raw.code),
        name: toNullableString(raw.name) ?? "Unknown room type",
        description: toNullableString(raw.description),
        capacity_adults: cap,
        capacity_children: null,
        max_occupancy: maxOcc ?? cap,
        is_active: toNullableBoolean(raw.active) ?? true,
        raw_status: null,
        source_updated_at: toNullableString(raw.updated_at),
    };
}
export function mapTesiproRoom(
    raw: TesiproRoomRaw,
    base: { orgId: string; propertyId: string; connectionId: string },
): CanonicalRoom {
    return {
        org_id: base.orgId,
        property_id: base.propertyId,
        connection_id: base.connectionId,
        external_room_id: toNullableString(raw.id ?? raw.room_id) ?? "",
        external_room_type_id: toNullableString(raw.room_type_id),
        room_number: toNullableString(raw.room_number),
        room_name: toNullableString(raw.room_name),
        floor: toNullableString(raw.floor),
        building: toNullableString(raw.building),
        is_active: toNullableBoolean(raw.active) ?? true,
    };
}

```

```

        operational_status: "UNKNOWN",
        raw_status: toNullableString(raw.status),
        source_updated_at: toNullableString(raw.updated_at),
    };
}
export function mapTesiproReservation(
    raw: TesiproReservationRaw,
    base: { orgId: string; propertyId: string; connectionId: string },
): CanonicalReservation {
    const rawStatus = toNullableString(raw.booking_status ?? raw.status);
    return {
        org_id: base.orgId,
        property_id: base.propertyId,
        connection_id: base.connectionId,
        external_reservation_id:
            toNullableString(raw.booking_id ?? raw.reservation_id) ?? "",
        external_confirmation_code: null,
        external_group_reservation_id: null,
        external_guest_id: null,
        external_primary_room_type_id: toNullableString(raw.room_type_id),
        external_assigned_room_id: toNullableString(raw.room_id),
        status: mapReservationStatus(rawStatus),
        raw_status: rawStatus,
        booked_at: toNullableString(raw.created_at),
        check_in_date: toNullableString(raw.arrival_date),
        check_out_date: toNullableString(raw.departure_date),
        nights: null,
        adults: toNullableNumber(raw.adults),
        children: toNullableNumber(raw.children),
        infants: null,
        rooms_count: 1,
        channel_code: null,
        channel_name: toNullableString(raw.channel),
        segment_code: null,
        segment_name: toNullableString(raw.segment),
        rate_plan_code: null,
        rate_plan_name: toNullableString(raw.rate_plan),
        currency_code: toNullableString(raw.currency),
        room_revenue_amount: null,
        total_amount: toNullableNumber(raw.total_amount),
        paid_amount: null,
        balance_amount: null,
        cancellation_at: null,
        source_updated_at: toNullableString(raw.updated_at),
    };
}

```

11. Repositorios

Aquí la clave es no meter SQL suelto por todos lados.

db.ts en __shared

```

// supabase/functions/_shared/db.ts
import { createClient } from "jsr:@supabase/supabase-js@2";
export function getServiceClient() {
    const url = Deno.env.get("SUPABASE_URL");
    const serviceRoleKey = Deno.env.get("SUPABASE_SERVICE_ROLE_KEY");
    if (!url || !serviceRoleKey) {

```

```

    throw new Error("Missing Supabase env vars");
  }
  return createClient(url, serviceRoleKey, {
    auth: { persistSession: false },
  });
}

```

PmsConnectionRepository.ts

```

// supabase/functions/_shared/pms/repositories/PmsConnectionRepository.ts
import { getServiceClient } from "../../db.ts";
import { PmsConnection } from "../../types/pms.types.ts";
export class PmsConnectionRepository {
  private db = getServiceClient();
  async getById(connectionId: string): Promise<PmsConnection> {
    const { data, error } = await this.db
      .from("pms_connections")
      .select("*")
      .eq("id", connectionId)
      .single();
    if (error || !data) {
      throw new Error(`PmsConnection not found: ${connectionId}`);
    }
    return data as PmsConnection;
  }
  async markSyncStarted(connectionId: string) {
    const { error } = await this.db
      .from("pms_connections")
      .update({
        last_sync_at: new Date().toISOString(),
        last_error_at: null,
        last_error_message: null,
      })
      .eq("id", connectionId);
    if (error) throw error;
  }
  async markSyncSuccess(connectionId: string) {
    const now = new Date().toISOString();
    const { error } = await this.db
      .from("pms_connections")
      .update({
        last_sync_at: now,
        last_success_sync_at: now,
        last_error_at: null,
        last_error_message: null,
        status: "ACTIVE",
      })
      .eq("id", connectionId);
    if (error) throw error;
  }
  async markSyncError(connectionId: string, message: string) {
    const { error } = await this.db
      .from("pms_connections")
      .update({
        last_error_at: new Date().toISOString(),
        last_error_message: message,
        status: "ERROR",
      })
      .eq("id", connectionId);
    if (error) throw error;
  }
}

```

PmsSyncJobRepository.ts

```
// supabase/functions/_shared/pms/repositories/PmsSyncJobRepository.ts
import { getServiceClient } from "../../db.ts";
import { PmsEntityType, PmsSyncMode } from "../../types/pms.types.ts";
export class PmsSyncJobRepository {
  private db = getServiceClient();
  async create(input: {
    org_id: string;
    property_id: string | null;
    connection_id: string;
    provider_code: string;
    entity_type: PmsEntityType;
    sync_mode: PmsSyncMode;
  }): Promise<string> {
    const { data, error } = await this.db
      .from("pms_sync_jobs")
      .insert({
        ...input,
        status: "RUNNING",
      })
      .select("id")
      .single();
    if (error || !data) throw error ?? new Error("Failed to create sync job");
    return data.id as string;
  }
  async finishSuccess(jobId: string, stats: {
    records_read: number;
    records_inserted: number;
    records_updated: number;
    records_failed: number;
  }) {
    const { error } = await this.db
      .from("pms_sync_jobs")
      .update({
        ...stats,
        status: "SUCCESS",
        finished_at: new Date().toISOString(),
      })
      .eq("id", jobId);
    if (error) throw error;
  }
  async finishFailed(jobId: string, errorSummary: string, stats?: Partial<{
    records_read: number;
    records_inserted: number;
    records_updated: number;
    records_failed: number;
  }>) {
    const { error } = await this.db
      .from("pms_sync_jobs")
      .update({
        ...stats,
        status: "FAILED",
        error_summary: errorSummary,
        finished_at: new Date().toISOString(),
      })
      .eq("id", jobId);
    if (error) throw error;
  }
}
```

PmsRawRecordRepository.ts

```
// supabase/functions/_shared/pms/repositories/PmsRawRecordRepository.ts
import { getServiceClient } from "../../db.ts";
import { PmsRawEnvelope } from "../../types/pms.types.ts";
import { sha256Hex } from "../../utils/hash.ts";
export class PmsRawRecordRepository {
  private db = getServiceClient();
  async insertMany(input: {
    orgId: string;
    propertyId: string | null;
    connectionId: string;
    syncJobId: string;
    rows: PmsRawEnvelope[];
  }) {
    if (!input.rows.length) return;
    const payload = [];
    for (const row of input.rows) {
      payload.push({
        org_id: input.orgId,
        property_id: input.propertyId,
        connection_id: input.connectionId,
        sync_job_id: input.syncJobId,
        provider_code: row.providerCode,
        entity_type: row.entityType,
        external_id: row.externalId,
        external_parent_id: row.externalParentId ?? null,
        external_updated_at: row.externalUpdatedAt ?? null,
        payload_json: row.payload,
        payload_hash: await sha256Hex(row.payload),
        last_seen_at: new Date().toISOString(),
      });
    }
    const { error } = await this.db.from("pms_raw_records").insert(payload);
    if (error) throw error;
  }
}
```

12. Repositorio canónico de reservas

Con uno vale como patrón. Luego replicas para room types, rooms, guests, stays.

PmsReservationRepository.ts

```
// supabase/functions/_shared/pms/repositories/PmsReservationRepository.ts
import { getServiceClient } from "../../db.ts";
import { CanonicalReservation } from "../../types/pms-canonical.types.ts";
export class PmsReservationRepository {
  private db = getServiceClient();
  async upsertMany(rows: CanonicalReservation[]): Promise<void> {
    if (!rows.length) return;
    const payload = rows.map((r) => ({
      org_id: r.org_id,
      property_id: r.property_id,
      connection_id: r.connection_id,
      external_reservation_id: r.external_reservation_id,
      external_confirmation_code: r.external_confirmation_code,
      external_group_reservation_id: r.external_group_reservation_id,
      status: r.status,
    }));
  }
}
```



```

        raw_status: r.raw_status,
        booked_at: r.booked_at,
        check_in_date: r.check_in_date,
        check_out_date: r.check_out_date,
        nights: r.nights,
        adults: r.adults,
        children: r.children,
        infants: r.infants,
        rooms_count: r.rooms_count,
        channel_code: r.channel_code,
        channel_name: r.channel_name,
        segment_code: r.segment_code,
        segment_name: r.segment_name,
        rate_plan_code: r.rate_plan_code,
        rate_plan_name: r.rate_plan_name,
        currency_code: r.currency_code,
        room_revenue_amount: r.room_revenue_amount,
        total_amount: r.total_amount,
        paid_amount: r.paid_amount,
        balance_amount: r.balance_amount,
        cancellation_at: r.cancellation_at,
        source_updated_at: r.source_updated_at,
    }));
    const { error } = await this.db
        .from("pms_reservations")
        .upsert(payload, {
            onConflict: "connection_id,external_reservation_id",
        });
    if (error) throw error;
}
}

```

13. Servicio de normalización

Este servicio decide qué mapper usar.

PmsNormalizationService.ts

```

// supabase/functions/_shared/pms/services/PmsNormalizationService.ts
import { PmsProviderCode } from "../../../types/pms.types.ts";
import {
    CanonicalReservation,
    CanonicalRoom,
    CanonicalRoomType,
} from "../../../types/pms-canonical.types.ts";
import {
    mapTesiproReservation,
    mapTesiproRoom,
    mapTesiproRoomType,
} from "../../../mappers/tesipro.mapper.ts";
export class PmsNormalizationService {
    static mapRoomType(
        provider: PmsProviderCode,
        raw: Record<string, unknown>,
        base: { orgId: string; propertyId: string; connectionId: string },
    ): CanonicalRoomType {
        switch (provider) {
            case "TESIPRO_ULYSES":
                return mapTesiproRoomType(raw, base);
            default:

```

```

        throw new Error(`RoomType mapper not implemented for provider ${
provider}`);
    }
}
static mapRoom(
    provider: PmsProviderCode,
    raw: Record<string, unknown>,
    base: { orgId: string; propertyId: string; connectionId: string },
): CanonicalRoom {
    switch (provider) {
        case "TESIPRO_ULYSES":
            return mapTesiproRoom(raw, base);
        default:
            throw new Error(`Room mapper not implemented for provider ${provider}`);
    }
}
static mapReservation(
    provider: PmsProviderCode,
    raw: Record<string, unknown>,
    base: { orgId: string; propertyId: string; connectionId: string },
): CanonicalReservation {
    switch (provider) {
        case "TESIPRO_ULYSES":
            return mapTesiproReservation(raw, base);
        default:
            throw new Error(`Reservation mapper not implemented for provider ${
provider}`);
    }
}
}
}

```

14. Servicio principal de sync

PmsSyncService.ts

```

// supabase/functions/_shared/pms/services/PmsSyncService.ts
import { PmsProviderFactory } from "../PmsProviderFactory.ts";
import { PmsConnectionRepository } from
    "../repositories/PmsConnectionRepository.ts";
import { PmsSyncJobRepository } from "../repositories/PmsSyncJobRepository.ts";
import { PmsRawRecordRepository } from
    "../repositories/PmsRawRecordRepository.ts";
import { PmsReservationRepository } from
    "../repositories/PmsReservationRepository.ts";
import { PmsNormalizationService } from "../PmsNormalizationService.ts";
import { PmsSyncRunInput, PmsRawEnvelope } from "../types/pms.types.ts";
export class PmsSyncService {
    private connections = new PmsConnectionRepository();
    private jobs = new PmsSyncJobRepository();
    private rawRepo = new PmsRawRecordRepository();
    private reservationRepo = new PmsReservationRepository();
    async run(input: PmsSyncRunInput) {
        const connection = await this.connections.getById(input.connectionId);
        const syncJobId = await this.jobs.create({
            org_id: connection.org_id,
            property_id: input.propertyId ?? connection.property_id,
            connection_id: connection.id,
            provider_code: connection.provider_code,
            entity_type: input.entityType,
            sync_mode: input.syncMode,

```

```

});
await this.connections.markSyncStarted(connection.id);
try {
  const connector = PmsProviderFactory.create(connection.provider_code);
  const ctx = {
    connection,
    entityType: input.entityType,
    syncMode: input.syncMode,
    propertyId: input.propertyId ?? connection.property_id,
    cursor: input.cursor ?? null,
    modifiedSince: input.modifiedSince ?? null,
    limit: input.limit ?? 100,
    syncJobId,
  };
  const paged = await connector.fetchByEntity(ctx);
  const rawRows: PmsRawEnvelope[] = paged.items.map((item) => ({
    providerCode: connection.provider_code,
    entityType: input.entityType,
    externalId:
      String(
        item["id"] ??
        item["reservation_id"] ??
        item["booking_id"] ??
        item["room_id"] ??
        item["guest_id"] ??
        crypto.randomUUID(),
      ),
    externalUpdatedAt: String(item["updated_at"] ?? "") || null,
    payload: item,
  }));
  await this.rawRepo.insertMany({
    orgId: connection.org_id,
    propertyId: ctx.propertyId,
    connectionId: connection.id,
    syncJobId,
    rows: rawRows,
  });
  if (input.entityType === "RESERVATION") {
    const canonical = paged.items.map((item) =>
      PmsNormalizationService.mapReservation(
        connection.provider_code,
        item,
        {
          orgId: connection.org_id,
          propertyId: ctx.propertyId ?? "",
          connectionId: connection.id,
        },
      ),
    );
    await this.reservationRepo.upsertMany(canonical);
  }
  await this.jobs.finishSuccess(syncJobId, {
    records_read: paged.items.length,
    records_inserted: paged.items.length,
    records_updated: 0,
    records_failed: 0,
  });
  await this.connections.markSyncSuccess(connection.id);
  return {
    ok: true,
    syncJobId,
    provider: connection.provider_code,
    entityType: input.entityType,
    read: paged.items.length,
  };
}

```

```

        nextCursor: paged.nextCursor,
    };
} catch (error) {
    const message = error instanceof Error ? error.message : "Unknown PMS sync
error";
    await this.jobs.finishFailed(syncJobId, message, {
        records_failed: 1,
    });
    await this.connections.markSyncError(connection.id, message);
    return {
        ok: false,
        syncJobId,
        error: message,
    };
}
}
}
}

```

15. Edge Function inicial

pms_sync_run/index.ts

```

// supabase/functions/pms_sync_run/index.ts
import "jsr:@supabase/functions-js/edge-runtime.d.ts";
import { preflight, json } from "../_shared/cors.ts";
import { requireUser } from "../_shared/auth.ts";
import { PmsSyncService } from "../_shared/pms/services/PmsSyncService.ts";
import { PmsSyncRunInput } from "../_shared/pms/types/pms.types.ts";
Deno.serve(async (req) => {
    if (req.method === "OPTIONS") return preflight(req);
    try {
        await requireUser(req);
        if (req.method !== "POST") {
            return json({ ok: false, error: "method_not_allowed" }, { status: 405 });
        }
        const body = (await req.json()) as PmsSyncRunInput;
        if (!body?.connectionId) {
            return json({ ok: false, error: "connectionId_required" }, { status:
400 });
        }
        if (!body?.entityType) {
            return json({ ok: false, error: "entityType_required" }, { status: 400 });
        }
        if (!body?.syncMode) {
            return json({ ok: false, error: "syncMode_required" }, { status: 400 });
        }
        const service = new PmsSyncService();
        const result = await service.run(body);
        return json(result, { status: result.ok ? 200 : 500 });
    } catch (error) {
        const message = error instanceof Error ? error.message : "unexpected_error";
        return json({ ok: false, error: message }, { status: 500 });
    }
});

```

16. Qué falta todavía

Esto ya es un paquete técnico base, pero no está terminado funcionalmente. Faltan estas piezas:

16.1 Repositorios equivalentes para:

- room types
- rooms
- guests
- stays
- folio summaries

16.2 Resolución de relaciones externas a internas

Por ejemplo:

- external_guest_id → guest_id
- external_room_type_id → room_type_id
- external_room_id → room_id

Eso debe hacerse en una segunda capa de enriquecimiento o en el upsert si ya existen.

16.3 Mappings persistidos en tabla

Ahora mismo el mapper está en código.

Lo correcto a medio plazo es usar:

- pms_status_mappings
- pms_channel_mappings
- pms_segment_mappings

16.4 Autenticación real Tesipro

No se puede cerrar hasta tener:

- documentación
 - endpoint base
 - auth model
 - formato de respuestas
-

17. Orden realista de implementación

Yo haría esto así, sin desorden:

Fase 1

Aplicar el SQL del modelo universal PMS.

Fase 2

Crear el paquete TS base:

- types
- connectors
- repositories
- services
- edge function

Fase 3

Cerrar primer flujo de una sola entidad:

- RESERVATION
o incluso mejor:
- ROOM_TYPE

La más fácil suele ser ROOM_TYPE.

Fase 4

Implementar persistencia real de:

- room types
- rooms
- reservations

Fase 5

Resolver relaciones:

- room_type_id
- room_id
- guest_id

Fase 6

Añadir admin UI interna para:

- crear conexión PMS
- ver estado
- lanzar sync manual
- ver jobs
- ver errores

CONTINUAMOS..

Te doy los archivos que faltan para completar la base técnica de la capa PMS, centrados en:

- PmsRoomTypeRepository
- PmsRoomRepository
- PmsGuestRepository
- PmsStayRepository
- ampliación de PmsSyncService para:
 - ROOM_TYPE
 - ROOM
 - GUEST
 - RESERVATION
 - STAY

Además, te añado una pequeña utilidad para resolver relaciones externas a internas sin meter todavía sobreingeniería.

1. PmsRoomTypeRepository.ts

```
// supabase/functions/_shared/pms/repositories/PmsRoomTypeRepository.ts
import { getServiceClient } from "../../db.ts";
import { CanonicalRoomType } from "../../types/pms-canonical.types.ts";
export class PmsRoomTypeRepository {
  private db = getServiceClient();
  async upsertMany(rows: CanonicalRoomType[]): Promise<void> {
    if (!rows.length) return;
    const payload = rows.map((r) => ({
      org_id: r.org_id,
      property_id: r.property_id,
      connection_id: r.connection_id,
```

```

        external_room_type_id: r.external_room_type_id,
        code: r.code,
        name: r.name,
        description: r.description,
        capacity_adults: r.capacity_adults,
        capacity_children: r.capacity_children,
        max_occupancy: r.max_occupancy,
        is_active: r.is_active,
        raw_status: r.raw_status,
        source_updated_at: r.source_updated_at,
    }));
    const { error } = await this.db
        .from("pms_room_types")
        .upsert(payload, {
            onConflict: "connection_id,external_room_type_id",
        });
    if (error) throw error;
}
async findInternalIdsByExternalIds(
    connectionId: string,
    externalIds: string[],
): Promise<Map<string, string>> {
    const cleaned = [...new Set(externalIds.filter(Boolean))];
    if (!cleaned.length) return new Map();
    const { data, error } = await this.db
        .from("pms_room_types")
        .select("id, external_room_type_id")
        .eq("connection_id", connectionId)
        .in("external_room_type_id", cleaned);
    if (error) throw error;
    const map = new Map<string, string>();
    for (const row of data ?? []) {
        map.set(row.external_room_type_id as string, row.id as string);
    }
    return map;
}
}

```

2. PmsRoomRepository.ts

```

// supabase/functions/_shared/pms/repositories/PmsRoomRepository.ts
import { getServiceClient } from "../../db.ts";
import { CanonicalRoom } from "../../types/pms-canonical.types.ts";
type RoomUpsertRow = CanonicalRoom & {
    room_type_id?: string | null;
};
export class PmsRoomRepository {
    private db = getServiceClient();
    async upsertMany(rows: RoomUpsertRow[]): Promise<void> {
        if (!rows.length) return;
        const payload = rows.map((r) => ({
            org_id: r.org_id,
            property_id: r.property_id,
            connection_id: r.connection_id,
            external_room_id: r.external_room_id,
            room_type_id: r.room_type_id ?? null,
            room_number: r.room_number,
            room_name: r.room_name,
            floor: r.floor,
            building: r.building,
            is_active: r.is_active,

```



```

        operational_status: r.operational_status,
        raw_status: r.raw_status,
        source_updated_at: r.source_updated_at,
    }));
    const { error } = await this.db
        .from("pms_rooms")
        .upsert(payload, {
            onConflict: "connection_id,external_room_id",
        });
    if (error) throw error;
}
async findInternalIdsByExternalIds(
    connectionId: string,
    externalIds: string[],
): Promise<Map<string, string>> {
    const cleaned = [...new Set(externalIds.filter(Boolean))];
    if (!cleaned.length) return new Map();
    const { data, error } = await this.db
        .from("pms_rooms")
        .select("id, external_room_id")
        .eq("connection_id", connectionId)
        .in("external_room_id", cleaned);
    if (error) throw error;
    const map = new Map<string, string>();
    for (const row of data ?? []) {
        map.set(row.external_room_id as string, row.id as string);
    }
    return map;
}
}

```

3. PmsGuestRepository.ts

```

// supabase/functions/_shared/pms/repositories/PmsGuestRepository.ts
import { getServiceClient } from "../../db.ts";
import { CanonicalGuest } from "../../types/pms-canonical.types.ts";
export class PmsGuestRepository {
    private db = getServiceClient();
    async upsertMany(rows: CanonicalGuest[]): Promise<void> {
        if (!rows.length) return;
        const payload = rows.map((r) => ({
            org_id: r.org_id,
            property_id: r.property_id,
            connection_id: r.connection_id,
            external_guest_id: r.external_guest_id,
            external_profile_id: r.external_profile_id,
            first_name: r.first_name,
            last_name: r.last_name,
            full_name: r.full_name,
            email: r.email,
            phone: r.phone,
            document_type: r.document_type,
            document_number: r.document_number,
            country_code: r.country_code,
            nationality_code: r.nationality_code,
            birth_date: r.birth_date,
            gender: r.gender,
            is_company: r.is_company,
            company_name: r.company_name,
            is_active: r.is_active,
            source_updated_at: r.source_updated_at,

```

```

    }));
    const { error } = await this.db
      .from("pms_guests")
      .upsert(payload, {
        onConflict: "connection_id,external_guest_id",
      });
    if (error) throw error;
  }
  async findInternalIdsByExternalIds(
    connectionId: string,
    externalIds: string[],
  ): Promise<Map<string, string>> {
    const cleaned = [...new Set(externalIds.filter(Boolean))];
    if (!cleaned.length) return new Map();
    const { data, error } = await this.db
      .from("pms_guests")
      .select("id, external_guest_id")
      .eq("connection_id", connectionId)
      .in("external_guest_id", cleaned);
    if (error) throw error;
    const map = new Map<string, string>();
    for (const row of data ?? []) {
      map.set(row.external_guest_id as string, row.id as string);
    }
    return map;
  }
}

```

4. PmsStayRepository.ts

```

// supabase/functions/_shared/pms/repositories/PmsStayRepository.ts
import { getServiceClient } from "../../db.ts";
import { CanonicalStay } from "../../types/pms-canonical.types.ts";
type StayUpsertRow = CanonicalStay & {
  reservation_id?: string | null;
  guest_id?: string | null;
  room_id?: string | null;
  room_type_id?: string | null;
};
export class PmsStayRepository {
  private db = getServiceClient();
  async upsertMany(rows: StayUpsertRow[]): Promise<void> {
    if (!rows.length) return;
    const payload = rows.map((r) => ({
      org_id: r.org_id,
      property_id: r.property_id,
      connection_id: r.connection_id,
      reservation_id: r.reservation_id ?? null,
      guest_id: r.guest_id ?? null,
      room_id: r.room_id ?? null,
      room_type_id: r.room_type_id ?? null,
      external_stay_id: r.external_stay_id,
      stay_status: r.stay_status,
      raw_status: r.raw_status,
      arrival_scheduled_at: r.arrival_scheduled_at,
      departure_scheduled_at: r.departure_scheduled_at,
      check_in_at: r.check_in_at,
      check_out_at: r.check_out_at,
      adults: r.adults,
      children: r.children,
      currency_code: r.currency_code,

```

```

        room_revenue_amount: r.room_revenue_amount,
        total_amount: r.total_amount,
        source_updated_at: r.source_updated_at,
    }));
    const { error } = await this.db
        .from("pms_stays")
        .upsert(payload, {
            onConflict: "connection_id,external_stay_id",
        });
    if (error) throw error;
}
}

```

5. PmsEntityResolver.ts

Esto te resuelve IDs internos a partir de IDs externos ya persistidos. Es simple, pero útil.

```

// supabase/functions/_shared/pms/services/PmsEntityResolver.ts
import { PmsGuestRepository } from "../../repositories/PmsGuestRepository.ts";
import { PmsRoomRepository } from "../../repositories/PmsRoomRepository.ts";
import { PmsRoomTypeRepository } from
    "../../repositories/PmsRoomTypeRepository.ts";
import { PmsReservationRepositoryLookup } from
    "../../repositories/PmsReservationRepositoryLookup.ts";
export class PmsEntityResolver {
    private guestRepo = new PmsGuestRepository();
    private roomRepo = new PmsRoomRepository();
    private roomTypeRepo = new PmsRoomTypeRepository();
    private reservationLookupRepo = new PmsReservationRepositoryLookup();
    async resolveGuests(connectionId: string, externalIds: string[]) {
        return await this.guestRepo.findInternalIdsByExternalIds(connectionId,
externalIds);
    }
    async resolveRooms(connectionId: string, externalIds: string[]) {
        return await this.roomRepo.findInternalIdsByExternalIds(connectionId,
externalIds);
    }
    async resolveRoomTypes(connectionId: string, externalIds: string[]) {
        return await this.roomTypeRepo.findInternalIdsByExternalIds(connectionId,
externalIds);
    }
    async resolveReservations(connectionId: string, externalIds: string[]) {
        return await
this.reservationLookupRepo.findInternalIdsByExternalIds(connectionId,
externalIds);
    }
}

```

6.

PmsReservationRepositoryLookup.ts

Como el repo de reservas que te di antes no incluía lookup, te lo separe para no tocar demasiado lo anterior.

```

// supabase/functions/_shared/pms/repositories/PmsReservationRepositoryLookup.ts
import { getServiceClient } from "../../db.ts";

```

```

export class PmsReservationRepositoryLookup {
  private db = getServiceClient();
  async findInternalIdsByExternalIds(
    connectionId: string,
    externalIds: string[],
  ): Promise<Map<string, string>> {
    const cleaned = [...new Set(externalIds.filter(Boolean))];
    if (!cleaned.length) return new Map();
    const { data, error } = await this.db
      .from("pms_reservations")
      .select("id, external_reservation_id")
      .eq("connection_id", connectionId)
      .in("external_reservation_id", cleaned);
    if (error) throw error;
    const map = new Map<string, string>();
    for (const row of data ?? []) {
      map.set(row.external_reservation_id as string, row.id as string);
    }
    return map;
  }
}

```

7. ampliación de PmsNormalizationService.ts

Aquí lo dejamos preparado también para guests y stays. Como Tesipro real no está cerrado, el mapping de guest y stay lo dejamos conservador.

```

// supabase/functions/_shared/pms/services/PmsNormalizationService.ts
import { PmsProviderCode } from "../types/pms.types.ts";
import {
  CanonicalGuest,
  CanonicalReservation,
  CanonicalRoom,
  CanonicalRoomType,
  CanonicalStay,
} from "../types/pms-canonical.types.ts";
import {
  mapTesiproGuest,
  mapTesiproReservation,
  mapTesiproRoom,
  mapTesiproRoomType,
  mapTesiproStay,
} from "../mappers/tesipro.mapper.ts";
export class PmsNormalizationService {
  static mapRoomType(
    provider: PmsProviderCode,
    raw: Record<string, unknown>,
    base: { orgId: string; propertyId: string; connectionId: string },
  ): CanonicalRoomType {
    switch (provider) {
      case "TESIPRO_ULYSES":
        return mapTesiproRoomType(raw, base);
      default:
        throw new Error(`RoomType mapper not implemented for provider $
{provider}`);
    }
  }
  static mapRoom(

```

```

    provider: PmsProviderCode,
    raw: Record<string, unknown>,
    base: { orgId: string; propertyId: string; connectionId: string },
  ): CanonicalRoom {
    switch (provider) {
      case "TESIPRO_ULYSES":
        return mapTesiproRoom(raw, base);
      default:
        throw new Error(`Room mapper not implemented for provider ${provider}`);
    }
  }
}
static mapGuest(
  provider: PmsProviderCode,
  raw: Record<string, unknown>,
  base: { orgId: string; propertyId: string | null; connectionId: string },
): CanonicalGuest {
  switch (provider) {
    case "TESIPRO_ULYSES":
      return mapTesiproGuest(raw, base);
    default:
      throw new Error(`Guest mapper not implemented for provider $
{provider}`);
  }
}
static mapReservation(
  provider: PmsProviderCode,
  raw: Record<string, unknown>,
  base: { orgId: string; propertyId: string; connectionId: string },
): CanonicalReservation {
  switch (provider) {
    case "TESIPRO_ULYSES":
      return mapTesiproReservation(raw, base);
    default:
      throw new Error(`Reservation mapper not implemented for provider $
{provider}`);
  }
}
static mapStay(
  provider: PmsProviderCode,
  raw: Record<string, unknown>,
  base: { orgId: string; propertyId: string; connectionId: string },
): CanonicalStay {
  switch (provider) {
    case "TESIPRO_ULYSES":
      return mapTesiproStay(raw, base);
    default:
      throw new Error(`Stay mapper not implemented for provider ${provider}`);
  }
}
}
}

```

8. ampliar tesipro.types.ts

```

// supabase/functions/_shared/pms/types/tesipro.types.ts
export interface TesiproGuestRaw {
  guest_id?: string;
  id?: string;
  profile_id?: string;
  first_name?: string;
  last_name?: string;
  full_name?: string;

```

```

email?: string;
phone?: string;
document_type?: string;
document_number?: string;
country?: string;
nationality?: string;
birth_date?: string;
gender?: string;
company_name?: string;
updated_at?: string;
}
export interface TesiproStayRaw {
  stay_id?: string;
  id?: string;
  reservation_id?: string;
  booking_id?: string;
  guest_id?: string;
  room_id?: string;
  room_type_id?: string;
  status?: string;
  arrival_scheduled_at?: string;
  departure_scheduled_at?: string;
  check_in_at?: string;
  check_out_at?: string;
  adults?: number | string;
  children?: number | string;
  total_amount?: number | string;
  room_revenue_amount?: number | string;
  currency?: string;
  updated_at?: string;
}

```

9. ampliar tesipro.mapper.ts

Te dejo el fichero completo, ya con guest y stay.

```

// supabase/functions/_shared/pms/mappers/tesipro.mapper.ts
import {
  CanonicalGuest,
  CanonicalReservation,
  CanonicalRoom,
  CanonicalRoomType,
  CanonicalStay,
} from "../types/pms-canonical.types.ts";
import {
  TesiproGuestRaw,
  TesiproReservationRaw,
  TesiproRoomRaw,
  TesiproRoomTypeRaw,
  TesiproStayRaw,
} from "../types/tesipro.types.ts";
import { mapReservationStatus } from "../reservationStatus.mapper.ts";
import { mapStayStatus } from "../stayStatus.mapper.ts";
function toNullableString(v: unknown): string | null {
  if (v === null || v === undefined) return null;
  const s = String(v).trim();
  return s.length ? s : null;
}
function toNullableNumber(v: unknown): number | null {
  if (v === null || v === undefined || v === "") return null;
  const n = Number(v);

```

```

    return Number.isFinite(n) ? n : null;
}
function toNullableBoolean(v: unknown): boolean | null {
  if (v === null || v === undefined) return null;
  if (typeof v === "boolean") return v;
  const s = String(v).trim().toLowerCase();
  if (["true", "1", "yes", "y", "si"].includes(s)) return true;
  if (["false", "0", "no", "n"].includes(s)) return false;
  return null;
}
export function mapTesiproRoomType(
  raw: TesiproRoomTypeRaw,
  base: { orgId: string; propertyId: string; connectionId: string },
): CanonicalRoomType {
  const cap = toNullableNumber(raw.capacity);
  const maxOcc = toNullableNumber(raw.max_occupancy);
  return {
    org_id: base.orgId,
    property_id: base.propertyId,
    connection_id: base.connectionId,
    external_room_type_id: toNullableString(raw.id) ?? "",
    code: toNullableString(raw.code),
    name: toNullableString(raw.name) ?? "Unknown room type",
    description: toNullableString(raw.description),
    capacity_adults: cap,
    capacity_children: null,
    max_occupancy: maxOcc ?? cap,
    is_active: toNullableBoolean(raw.active) ?? true,
    raw_status: null,
    source_updated_at: toNullableString(raw.updated_at),
  };
}
export function mapTesiproRoom(
  raw: TesiproRoomRaw,
  base: { orgId: string; propertyId: string; connectionId: string },
): CanonicalRoom {
  return {
    org_id: base.orgId,
    property_id: base.propertyId,
    connection_id: base.connectionId,
    external_room_id: toNullableString(raw.id ?? raw.room_id) ?? "",
    external_room_type_id: toNullableString(raw.room_type_id),
    room_number: toNullableString(raw.room_number),
    room_name: toNullableString(raw.room_name),
    floor: toNullableString(raw.floor),
    building: toNullableString(raw.building),
    is_active: toNullableBoolean(raw.active) ?? true,
    operational_status: "UNKNOWN",
    raw_status: toNullableString(raw.status),
    source_updated_at: toNullableString(raw.updated_at),
  };
}
export function mapTesiproGuest(
  raw: TesiproGuestRaw,
  base: { orgId: string; propertyId: string | null; connectionId: string },
): CanonicalGuest {
  const firstName = toNullableString(raw.first_name);
  const lastName = toNullableString(raw.last_name);
  return {
    org_id: base.orgId,
    property_id: base.propertyId,
    connection_id: base.connectionId,
    external_guest_id: toNullableString(raw.guest_id ?? raw.id) ?? "",
    external_profile_id: toNullableString(raw.profile_id),
  };
}

```

```

    first_name: firstName,
    last_name: lastName,
    full_name:
      toNullableString(raw.full_name) ??
      [firstName, lastName].filter(Boolean).join(" ") ||
      null,
    email: toNullableString(raw.email),
    phone: toNullableString(raw.phone),
    document_type: toNullableString(raw.document_type),
    document_number: toNullableString(raw.document_number),
    country_code: toNullableString(raw.country),
    nationality_code: toNullableString(raw.nationality),
    birth_date: toNullableString(raw.birth_date),
    gender: toNullableString(raw.gender),
    is_company: null,
    company_name: toNullableString(raw.company_name),
    is_active: true,
    source_updated_at: toNullableString(raw.updated_at),
  };
}
export function mapTesiproReservation(
  raw: TesiproReservationRaw,
  base: { orgId: string; propertyId: string; connectionId: string },
): CanonicalReservation {
  const rawStatus = toNullableString(raw.booking_status ?? raw.status);
  return {
    org_id: base.orgId,
    property_id: base.propertyId,
    connection_id: base.connectionId,
    external_reservation_id:
      toNullableString(raw.booking_id ?? raw.reservation_id) ?? "",
    external_confirmation_code: null,
    external_group_reservation_id: null,
    external_guest_id: null,
    external_primary_room_type_id: toNullableString(raw.room_type_id),
    external_assigned_room_id: toNullableString(raw.room_id),
    status: mapReservationStatus(rawStatus),
    raw_status: rawStatus,
    booked_at: toNullableString(raw.created_at),
    check_in_date: toNullableString(raw.arrival_date),
    check_out_date: toNullableString(raw.departure_date),
    nights: null,
    adults: toNullableNumber(raw.adults),
    children: toNullableNumber(raw.children),
    infants: null,
    rooms_count: 1,
    channel_code: null,
    channel_name: toNullableString(raw.channel),
    segment_code: null,
    segment_name: toNullableString(raw.segment),
    rate_plan_code: null,
    rate_plan_name: toNullableString(raw.rate_plan),
    currency_code: toNullableString(raw.currency),
    room_revenue_amount: null,
    total_amount: toNullableNumber(raw.total_amount),
    paid_amount: null,
    balance_amount: null,
    cancellation_at: null,
    source_updated_at: toNullableString(raw.updated_at),
  };
}
export function mapTesiproStay(
  raw: TesiproStayRaw,
  base: { orgId: string; propertyId: string; connectionId: string },

```



```

): CanonicalStay {
  const rawStatus = toNullableString(raw.status);
  return {
    org_id: base.orgId,
    property_id: base.propertyId,
    connection_id: base.connectionId,
    external_stay_id: toNullableString(raw.stay_id ?? raw.id),
    external_reservation_id: toNullableString(raw.reservation_id ??
raw.booking_id),
    external_guest_id: toNullableString(raw.guest_id),
    external_room_id: toNullableString(raw.room_id),
    external_room_type_id: toNullableString(raw.room_type_id),
    stay_status: mapStayStatus(rawStatus),
    raw_status: rawStatus,
    arrival_scheduled_at: toNullableString(raw.arrival_scheduled_at),
    departure_scheduled_at: toNullableString(raw.departure_scheduled_at),
    check_in_at: toNullableString(raw.check_in_at),
    check_out_at: toNullableString(raw.check_out_at),
    adults: toNullableNumber(raw.adults),
    children: toNullableNumber(raw.children),
    currency_code: toNullableString(raw.currency),
    room_revenue_amount: toNullableNumber(raw.room_revenue_amount),
    total_amount: toNullableNumber(raw.total_amount),
    source_updated_at: toNullableString(raw.updated_at),
  };
}

```

10. PmsSyncService.ts ampliado

Este ya te vale como núcleo serio inicial.

```

// supabase/functions/_shared/pms/services/PmsSyncService.ts
import { PmsProviderFactory } from "../PmsProviderFactory.ts";
import { PmsConnectionRepository } from
"../repositories/PmsConnectionRepository.ts";
import { PmsSyncJobRepository } from "../repositories/PmsSyncJobRepository.ts";
import { PmsRawRecordRepository } from
"../repositories/PmsRawRecordRepository.ts";
import { PmsReservationRepository } from
"../repositories/PmsReservationRepository.ts";
import { PmsRoomTypeRepository } from
"../repositories/PmsRoomTypeRepository.ts";
import { PmsRoomRepository } from "../repositories/PmsRoomRepository.ts";
import { PmsGuestRepository } from "../repositories/PmsGuestRepository.ts";
import { PmsStayRepository } from "../repositories/PmsStayRepository.ts";
import { PmsNormalizationService } from "../PmsNormalizationService.ts";
import { PmsEntityResolver } from "../PmsEntityResolver.ts";
import { PmsRawEnvelope, PmsSyncRunInput } from "../types/pms.types.ts";
function buildExternalId(item: Record<string, unknown>): string {
  const candidates = [
    item["id"],
    item["reservation_id"],
    item["booking_id"],
    item["stay_id"],
    item["room_id"],
    item["guest_id"],
    item["profile_id"],
  ];
  for (const c of candidates) {
    if (c !== undefined && c !== null && String(c).trim()) {
      return String(c);
    }
  }
}

```

```

    }
  }
  return crypto.randomUUID();
}
export class PmsSyncService {
  private connections = new PmsConnectionRepository();
  private jobs = new PmsSyncJobRepository();
  private rawRepo = new PmsRawRecordRepository();
  private roomTypeRepo = new PmsRoomTypeRepository();
  private roomRepo = new PmsRoomRepository();
  private guestRepo = new PmsGuestRepository();
  private reservationRepo = new PmsReservationRepository();
  private stayRepo = new PmsStayRepository();
  private resolver = new PmsEntityResolver();
  async run(input: PmsSyncRunInput) {
    const connection = await this.connections.getById(input.connectionId);
    const effectivePropertyId = input.propertyId ?? connection.property_id;
    if (
      ["ROOM_TYPE", "ROOM", "RESERVATION", "STAY"].includes(input.entityType) &&
      !effectivePropertyId
    ) {
      throw new Error(`property_id is required for entity ${input.entityType}`);
    }
    const syncJobId = await this.jobs.create({
      org_id: connection.org_id,
      property_id: effectivePropertyId,
      connection_id: connection.id,
      provider_code: connection.provider_code,
      entity_type: input.entityType,
      sync_mode: input.syncMode,
    });
    await this.connections.markSyncStarted(connection.id);
    try {
      const connector = PmsProviderFactory.create(connection.provider_code);
      const ctx = {
        connection,
        entityType: input.entityType,
        syncMode: input.syncMode,
        propertyId: effectivePropertyId,
        cursor: input.cursor ?? null,
        modifiedSince: input.modifiedSince ?? null,
        limit: input.limit ?? 100,
        syncJobId,
      };
      const paged = await connector.fetchByEntity(ctx);
      const rawRows: PmsRawEnvelope[] = paged.items.map((item) => ({
        providerCode: connection.provider_code,
        entityType: input.entityType,
        externalId: buildExternalId(item),
        externalUpdatedAt:
          item["updated_at"] !== undefined && item["updated_at"] !== null
            ? String(item["updated_at"])
            : null,
        payload: item,
      }));
      await this.rawRepo.insertMany({
        orgId: connection.org_id,
        propertyId: effectivePropertyId,
        connectionId: connection.id,
        syncJobId,
        rows: rawRows,
      });
      let recordsInserted = 0;
      let recordsUpdated = 0;
    }
  }
}

```

```

switch (input.entityType) {
  case "ROOM_TYPE": {
    const canonical = paged.items.map((item) =>
      PmsNormalizationService.mapRoomType(connection.provider_code, item,
{
      orgId: connection.org_id,
      propertyId: effectivePropertyId!,
      connectionId: connection.id,
    })),
    );
    await this.roomTypeRepo.upsertMany(canonical);
    recordsInserted = canonical.length;
    break;
  }
  case "ROOM": {
    const canonical = paged.items.map((item) =>
      PmsNormalizationService.mapRoom(connection.provider_code, item, {
        orgId: connection.org_id,
        propertyId: effectivePropertyId!,
        connectionId: connection.id,
      })),
    );
    const roomTypeExternalIds = canonical
      .map((r) => r.external_room_type_id ?? "")
      .filter(Boolean);
    const roomTypeMap = await this.resolver.resolveRoomTypes(
      connection.id,
      roomTypeExternalIds,
    );
    const rows = canonical.map((r) => ({
      ...r,
      room_type_id: r.external_room_type_id
        ? roomTypeMap.get(r.external_room_type_id) ?? null
        : null,
    }));
    await this.roomRepo.upsertMany(rows);
    recordsInserted = rows.length;
    break;
  }
  case "GUEST": {
    const canonical = paged.items.map((item) =>
      PmsNormalizationService.mapGuest(connection.provider_code, item, {
        orgId: connection.org_id,
        propertyId: effectivePropertyId ?? null,
        connectionId: connection.id,
      })),
    );
    await this.guestRepo.upsertMany(canonical);
    recordsInserted = canonical.length;
    break;
  }
  case "RESERVATION": {
    const canonical = paged.items.map((item) =>
      PmsNormalizationService.mapReservation(connection.provider_code,
item, {
        orgId: connection.org_id,
        propertyId: effectivePropertyId!,
        connectionId: connection.id,
      })),
    );
    const guestMap = await this.resolver.resolveGuests(
      connection.id,
      canonical.map((r) => r.external_guest_id ?? "").filter(Boolean),
    );
  }
}

```

```

        const roomTypeMap = await this.resolver.resolveRoomTypes(
            connection.id,
            canonical.map((r) => r.external_primary_room_type_id ??
""").filter(Boolean),
        );
        const roomMap = await this.resolver.resolveRooms(
            connection.id,
            canonical.map((r) => r.external_assigned_room_id ??
""").filter(Boolean),
        );
        const rows = canonical.map((r) => ({
            ...r,
            guest_id: r.external_guest_id
                ? guestMap.get(r.external_guest_id) ?? null
                : null,
            primary_room_type_id: r.external_primary_room_type_id
                ? roomTypeMap.get(r.external_primary_room_type_id) ?? null
                : null,
            assigned_room_id: r.external_assigned_room_id
                ? roomMap.get(r.external_assigned_room_id) ?? null
                : null,
        }));
        await this.reservationRepo.upsertMany(rows);
        recordsInserted = rows.length;
        break;
    }
    case "STAY": {
        const canonical = paged.items.map((item) =>
            PmsNormalizationService.mapStay(connection.provider_code, item, {
                orgId: connection.org_id,
                propertyId: effectivePropertyId!,
                connectionId: connection.id,
            })),
        );
        const reservationMap = await this.resolver.resolveReservations(
            connection.id,
            canonical.map((s) => s.external_reservation_id ??
""").filter(Boolean),
        );
        const guestMap = await this.resolver.resolveGuests(
            connection.id,
            canonical.map((s) => s.external_guest_id ?? "").filter(Boolean),
        );
        const roomMap = await this.resolver.resolveRooms(
            connection.id,
            canonical.map((s) => s.external_room_id ?? "").filter(Boolean),
        );
        const roomTypeMap = await this.resolver.resolveRoomTypes(
            connection.id,
            canonical.map((s) => s.external_room_type_id ?? "").filter(Boolean),
        );
        const rows = canonical.map((s) => ({
            ...s,
            reservation_id: s.external_reservation_id
                ? reservationMap.get(s.external_reservation_id) ?? null
                : null,
            guest_id: s.external_guest_id
                ? guestMap.get(s.external_guest_id) ?? null
                : null,
            room_id: s.external_room_id
                ? roomMap.get(s.external_room_id) ?? null
                : null,
            room_type_id: s.external_room_type_id
                ? roomTypeMap.get(s.external_room_type_id) ?? null

```

```

        : null,
      }));
      await this.stayRepo.upsertMany(rows);
      recordsInserted = rows.length;
      break;
    }
    default:
      throw new Error(`Entity type not implemented in sync service: ${input.entityType}`);
  }
  await this.jobs.finishSuccess(syncJobId, {
    records_read: paged.items.length,
    records_inserted: recordsInserted,
    records_updated: recordsUpdated,
    records_failed: 0,
  });
  await this.connections.markSyncSuccess(connection.id);
  return {
    ok: true,
    syncJobId,
    provider: connection.provider_code,
    entityType: input.entityType,
    read: paged.items.length,
    inserted: recordsInserted,
    updated: recordsUpdated,
    nextCursor: paged.nextCursor,
  };
} catch (error) {
  const message = error instanceof Error ? error.message : "Unknown PMS sync error";
  await this.jobs.finishFailed(syncJobId, message, {
    records_failed: 1,
  });
  await this.connections.markSyncError(connection.id, message);
  return {
    ok: false,
    syncJobId,
    error: message,
  };
}
}
}
}

```

11. corrección importante en PmsReservationRepository.ts

El repo que te pasé antes no aceptaba campos resueltos como `guest_id`, `primary_room_type_id`, `assigned_room_id`. Debe quedar así:

```

// supabase/functions/_shared/pms/repositories/PmsReservationRepository.ts
import { getServiceClient } from "../../db.ts";
import { CanonicalReservation } from "../../../types/pms-canonical.types.ts";
type ReservationUpsertRow = CanonicalReservation & {
  guest_id?: string | null;
  primary_room_type_id?: string | null;
  assigned_room_id?: string | null;
};
export class PmsReservationRepository {
  private db = getServiceClient();

```

```

async upsertMany(rows: ReservationUpsertRow[]): Promise<void> {
  if (!rows.length) return;
  const payload = rows.map((r) => ({
    org_id: r.org_id,
    property_id: r.property_id,
    connection_id: r.connection_id,
    external_reservation_id: r.external_reservation_id,
    external_confirmation_code: r.external_confirmation_code,
    external_group_reservation_id: r.external_group_reservation_id,
    guest_id: r.guest_id ?? null,
    primary_room_type_id: r.primary_room_type_id ?? null,
    assigned_room_id: r.assigned_room_id ?? null,
    status: r.status,
    raw_status: r.raw_status,
    booked_at: r.booked_at,
    check_in_date: r.check_in_date,
    check_out_date: r.check_out_date,
    nights: r.nights,
    adults: r.adults,
    children: r.children,
    infants: r.infants,
    rooms_count: r.rooms_count,
    channel_code: r.channel_code,
    channel_name: r.channel_name,
    segment_code: r.segment_code,
    segment_name: r.segment_name,
    rate_plan_code: r.rate_plan_code,
    rate_plan_name: r.rate_plan_name,
    currency_code: r.currency_code,
    room_revenue_amount: r.room_revenue_amount,
    total_amount: r.total_amount,
    paid_amount: r.paid_amount,
    balance_amount: r.balance_amount,
    cancellation_at: r.cancellation_at,
    source_updated_at: r.source_updated_at,
  }));
  const { error } = await this.db
    .from("pms_reservations")
    .upsert(payload, {
      onConflict: "connection_id,external_reservation_id",
    });
  if (error) throw error;
}
}

```

12. lo que debes probar primero

Antes de tocar guests o stays, yo probaría solo esto:

orden correcto de sync manual

1. ROOM_TYPE
2. ROOM
3. GUEST
4. RESERVATION

5. STAY

Porque si lanzas reservas antes de habitaciones o room types, las relaciones llegarán parcialmente vacías. No rompe nada, pero tendrás menos enriquecimiento.

13. lo que sigue faltando, sin engaños

Esto todavía no es un conector PMS completo. Sigue faltando:

- implementación real de TesiproConnector
- autenticación real
- paginación real
- sync incremental de verdad
- resolver `records_updated` vs `records_inserted`
- upsert de `pms_reservation_guests`
- soporte para FOLIO
- mappings persistidos en DB
- UI de administración

Pero como base técnica, ya es seria y bastante limpia.

La idea es que puedas:

1. crear un proveedor si no existe,
2. crear una conexión PMS de prueba,
3. asociarla a una propiedad,
4. lanzar la Edge Function `pms_sync_run`,
5. ver jobs, errores y registros raw/canónicos.

Voy a darte un bloque realista y usable, pero con una advertencia importante:

Antes de ejecutar esto

Tu tabla `pms_connections` exige `org_id` y en muchos casos también usarás un `property_id` real de tu sistema. Por tanto, **tienes que sustituir los UUID de ejemplo** por los de una organización y una propiedad que ya existan en tu base.

No voy a inventarme nombres de tablas de organizaciones o propiedades de Debaqu que no has confirmado para esta parte. Así evitamos romper algo.

1) SQL SEED BÁSICO

proveedor + conexión + property mapping

Sustituye estos valores:

- YOUR_ORG_ID
- YOUR_PROPERTY_ID

```
-- =====
-- SEED BÁSICO PMS - DEBACU
-- Crear proveedor, conexión y mapping de propiedad
-- =====
-- 1. Asegurar proveedor TESIPRO_ULYSES
insert into pms_providers (
  code,
  name,
  active,
  auth_type,
  supports_webhooks,
  supports_multi_property
)
values (
  'TESIPRO_ULYSES',
  'Tesipro Ulyses PMS',
  true,
  'CUSTOM',
  false,
  true
)
on conflict (code) do update
set
  name = excluded.name,
  active = excluded.active,
  auth_type = excluded.auth_type,
  supports_webhooks = excluded.supports_webhooks,
  supports_multi_property = excluded.supports_multi_property;
-- 2. Crear conexión de prueba
-- OJO: sustituir YOUR_ORG_ID y YOUR_PROPERTY_ID
insert into pms_connections (
  id,
  org_id,
  property_id,
  provider_code,
  environment,
  external_account_id,
  external_group_id,
  auth_mode,
  status
)
values (
  gen_random_uuid(),
  'YOUR_ORG_ID'::uuid,
  'YOUR_PROPERTY_ID'::uuid,
  'TESIPRO_ULYSES',
  'sandbox',
  'tesipro-demo-account',
  'tesipro-demo-group',
  'CUSTOM',
  'ACTIVE'
);
```



```

    'PENDING'
)
returning id;
-- 3. Crear mapping de propiedad
-- IMPORTANTE:
-- aquí debes usar el id devuelto por el insert anterior
-- o bien repetir la búsqueda por org/property/provider
insert into pms_property_mappings (
    id,
    org_id,
    property_id,
    connection_id,
    external_property_id,
    external_property_code,
    external_property_name,
    is_primary
)
select
    gen_random_uuid(),
    c.org_id,
    c.property_id,
    c.id,
    'TESIPRO_PROPERTY_001',
    'TESI001',
    'Tesipro Demo Property',
    true
from pms_connections c
where c.org_id = 'YOUR_ORG_ID'::uuid
    and c.property_id = 'YOUR_PROPERTY_ID'::uuid
    and c.provider_code = 'TESIPRO_ULYSES'
order by c.created_at desc
limit 1;

```

2) SQL MÁS LIMPIO

versión con bloque DO \$\$ para no andar copiando el connection_id

Esta versión es mejor porque crea todo de una vez y controla duplicados mínimos.

```

do $$
declare
    v_org_id uuid := 'YOUR_ORG_ID'::uuid;
    v_property_id uuid := 'YOUR_PROPERTY_ID'::uuid;
    v_connection_id uuid;
begin
    -- Asegurar provider
    insert into pms_providers (
        code,
        name,
        active,
        auth_type,
        supports_webhooks,
        supports_multi_property
    )
    values (
        'TESIPRO_ULYSES',

```

```

    'Tesipro Ulyses PMS',
    true,
    'CUSTOM',
    false,
    true
)
on conflict (code) do update
set
    name = excluded.name,
    active = excluded.active,
    auth_type = excluded.auth_type,
    supports_webhooks = excluded.supports_webhooks,
    supports_multi_property = excluded.supports_multi_property;
-- Reutilizar conexión si ya existe
select id
    into v_connection_id
from pms_connections
where org_id = v_org_id
    and property_id = v_property_id
    and provider_code = 'TESIPRO_ULYSES'
    and environment = 'sandbox'
order by created_at desc
limit 1;
-- Si no existe, crearla
if v_connection_id is null then
    insert into pms_connections (
        id,
        org_id,
        property_id,
        provider_code,
        environment,
        external_account_id,
        external_group_id,
        auth_mode,
        status
    )
    values (
        gen_random_uuid(),
        v_org_id,
        v_property_id,
        'TESIPRO_ULYSES',
        'sandbox',
        'tesipro-demo-account',
        'tesipro-demo-group',
        'CUSTOM',
        'PENDING'
    )
    returning id into v_connection_id;
end if;
-- Mapping de propiedad
insert into pms_property_mappings (
    id,
    org_id,
    property_id,
    connection_id,
    external_property_id,
    external_property_code,
    external_property_name,
    is_primary
)
values (
    gen_random_uuid(),
    v_org_id,
    v_property_id,

```

```
v_connection_id,  
'TESIPRO_PROPERTY_001',  
'TESI001',  
'Tesipro Demo Property',  
true  
)  
on conflict (connection_id, external_property_id) do nothing;  
raise notice 'PMS connection ready: %', v_connection_id;  
end $$;
```

3) SQL PARA VER LO QUE HAS CREADO

Ver conexiones PMS

```
select  
  id,  
  org_id,  
  property_id,  
  provider_code,  
  environment,  
  auth_mode,  
  status,  
  last_sync_at,  
  last_success_sync_at,  
  last_error_at,  
  last_error_message,  
  created_at  
from pms_connections  
order by created_at desc;
```

Ver mapping de propiedad

```
select  
  id,  
  org_id,  
  property_id,  
  connection_id,  
  external_property_id,  
  external_property_code,  
  external_property_name,  
  is_primary,  
  created_at  
from pms_property_mappings  
order by created_at desc;
```

Ver jobs PMS

```
select  
  id,  
  org_id,  
  property_id,  
  connection_id,  
  provider_code,  
  entity_type,  
  sync_mode,  
  status,  
  records_read,
```

```
records_inserted,  
records_updated,  
records_failed,  
error_summary,  
started_at,  
finished_at  
from pms_sync_jobs  
order by started_at desc;
```

Ver raw records

```
select  
  id,  
  connection_id,  
  entity_type,  
  external_id,  
  external_updated_at,  
  first_seen_at,  
  last_seen_at,  
  created_at  
from pms_raw_records  
order by created_at desc  
limit 100;
```

Ver reservas canónicas

```
select  
  id,  
  org_id,  
  property_id,  
  connection_id,  
  external_reservation_id,  
  status,  
  raw_status,  
  check_in_date,  
  check_out_date,  
  total_amount,  
  source_updated_at,  
  created_at,  
  updated_at  
from pms_reservations  
order by updated_at desc  
limit 100;
```

4) CÓMO LANZAR EL PRIMER TEST DE `pms_sync_run`

Como el `TesiproConnector` todavía está en placeholder y lanza error, el primer test útil ahora mismo es comprobar que:

- la conexión existe,
- la Edge Function responde,
- se crea `pms_sync_job`,
- queda trazado el error en `pms_connections`,

- no rompe nada.

Payload ejemplo

```
{
  "connectionId": "PON_AQUI_EL_CONNECTION_ID",
  "entityType": "ROOM_TYPE",
  "syncMode": "FULL",
  "propertyId": "YOUR_PROPERTY_ID",
  "limit": 100
}
```

5) EJEMPLO DE curl

Ajusta URL y token.

```
curl -X POST "https://YOUR_PROJECT_REF.supabase.co/functions/v1/pms_sync_run" \
-H "Authorization: Bearer YOUR_USER_JWT" \
-H "Content-Type: application/json" \
-d '{
  "connectionId": "PON_AQUI_EL_CONNECTION_ID",
  "entityType": "ROOM_TYPE",
  "syncMode": "FULL",
  "propertyId": "YOUR_PROPERTY_ID",
  "limit": 100
}'
```

6) QUÉ DEBERÍA PASAR AHORA MISMO

Con el código actual, como el `TesiproConnector` no está implementado, lo esperable es:

- se crea un `pms_sync_job`
- estado final del job: `FAILED`
- `pms_connections.status` pasa a `ERROR`
- `last_error_message` tendrá algo tipo:
 - `Tesipro fetchRoomTypes pending API validation`
 - o similar

Eso **no es un fallo malo**. Al contrario: significa que la arquitectura base ya está viva y el circuito de error funciona.

7) SQL PARA VER EL ERROR DE LA PRUEBA

```
select
  id,
  provider_code,
  status,
  last_sync_at,
  last_success_sync_at,
  last_error_at,
  last_error_message
from pms_connections
where provider_code = 'TESIPRO_ULYSES'
order by updated_at desc nulls last;
select
  id,
  connection_id,
  entity_type,
  sync_mode,
  status,
  error_summary,
  records_read,
  records_inserted,
  records_updated,
  records_failed,
  started_at,
  finished_at
from pms_sync_jobs
where provider_code = 'TESIPRO_ULYSES'
order by started_at desc;
```

8) SEED OPCIONAL DE MAPPINGS DE ESTADO

Esto no es imprescindible aún, pero ya te deja preparada la tabla.

```
insert into pms_status_mappings (
  id,
  provider_code,
  entity_type,
  raw_value,
  canonical_value,
  is_active
)
values
  (gen_random_uuid(), 'TESIPRO_ULYSES', 'RESERVATION', 'confirmed', 'CONFIRMED',
  true),
  (gen_random_uuid(), 'TESIPRO_ULYSES', 'RESERVATION', 'booked', 'CONFIRMED',
  true),
  (gen_random_uuid(), 'TESIPRO_ULYSES', 'RESERVATION', 'cancelled', 'CANCELLED',
  true),
  (gen_random_uuid(), 'TESIPRO_ULYSES', 'RESERVATION', 'inhouse', 'IN_HOUSE',
  true),
  (gen_random_uuid(), 'TESIPRO_ULYSES', 'STAY', 'expected', 'EXPECTED', true),
  (gen_random_uuid(), 'TESIPRO_ULYSES', 'STAY', 'inhouse', 'IN_HOUSE', true),
  (gen_random_uuid(), 'TESIPRO_ULYSES', 'STAY', 'checked_out', 'CHECKED_OUT',
  true),
```

```
(gen_random_uuid(), 'TESIPRO_ULYSES', 'ROOM', 'clean', 'CLEAN', true),  
(gen_random_uuid(), 'TESIPRO_ULYSES', 'ROOM', 'dirty', 'DIRTY', true),  
(gen_random_uuid(), 'TESIPRO_ULYSES', 'ROOM', 'out_of_service',  
'OUT_OF_SERVICE', true)  
on conflict (provider_code, entity_type, raw_value) do nothing;
```

9) COMPROBACIÓN RÁPIDA ANTES DE EJECUTAR

Necesitas confirmar estas tres cosas:

A. que el `org_id` existe

```
select 'YOUR_ORG_ID'::uuid;
```

B. que el `property_id` existe

```
select 'YOUR_PROPERTY_ID'::uuid;
```

C. que esa propiedad pertenece realmente a esa organización

Aquí depende de tu modelo real de Debacu. No te doy SQL inventado porque no has confirmado la tabla concreta de propiedades para este módulo. Si ya tienes una tabla tipo `debacu_eval_properties`, entonces conviene comprobarlo ahí.

10) CONSEJO PRÁCTICO

Para la primera prueba, usa **una sola propiedad** y una sola conexión.
No montes ya multi-property automático hasta que esté claro:

- cómo autentica Tesipro,
- si el acceso es por grupo o por propiedad,
- y qué devuelve realmente la API.

Si intentas resolver eso sin documentación real, te complicas solo.

Seguimos....

Si todavía no tenemos la API real de Tesipro cerrada, lo más útil es montar un modo mock dentro de `TesiproConnector` para poder probar ya:

- creación de `sync_job`
- lectura de datos

- persistencia raw
- normalización
- upsert canónico
- resolución de relaciones
- validación del pipeline end-to-end

Sin eso, la arquitectura existe, pero no la estás ejercitando de verdad.

Voy a dejarte una versión práctica y simple.

Objetivo del modo mock

Permitir que `TesiproConnector` funcione en dos modos:

modo real

cuando ya tengas:

- `TESIPRO_API_BASE_URL`
- auth real
- endpoints reales

modo mock

cuando quieras simular datos PMS y probar Debacu sin depender de Tesipro.

Estrategia recomendada

Usar una variable de entorno:

`PMS MOCK MODE=true`

y opcionalmente:

`PMS MOCK PROVIDER=TESIPRO_ULYSES`

Cuando eso esté activo, `TesiproConnector` devolverá datos simulados para:

- `ROOM_TYPE`
- `ROOM`
- `GUEST`
- `RESERVATION`
- `STAY`

1) archivo mock de Tesipro

supabase/functions/_shared/pms/mocks/ tesipro.mock.ts

```
// supabase/functions/_shared/pms/mocks/tesipro.mock.ts
import { PmsPagedResult } from "../../types/pms.types.ts";
export function getMockTesiproRoomTypes(): PmsPagedResult<Record<string,
unknown>> {
  return {
    items: [
      {
        id: "RT_STD",
        code: "STD",
        name: "Standard Double",
        description: "Standard double room",
        capacity: 2,
        max_occupancy: 2,
        active: true,
        updated_at: "2026-03-16T10:00:00Z",
      },
      {
        id: "RT_SUP",
        code: "SUP",
        name: "Superior Double",
        description: "Superior room with terrace",
        capacity: 2,
        max_occupancy: 3,
        active: true,
        updated_at: "2026-03-16T10:00:00Z",
      },
      {
        id: "RT_FAM",
        code: "FAM",
        name: "Family Room",
        description: "Family room",
        capacity: 2,
        max_occupancy: 4,
        active: true,
        updated_at: "2026-03-16T10:00:00Z",
      },
    ],
    nextCursor: null,
    sourceUpdatedAtMax: "2026-03-16T10:00:00Z",
  };
}
export function getMockTesiproRooms(): PmsPagedResult<Record<string, unknown>> {
  return {
    items: [
      {
        id: "ROOM_101",
        room_number: "101",
        room_name: "Room 101",
        room_type_id: "RT_STD",
        floor: "1",
        building: "Main",
        status: "clean",
        active: true,
      },
    ],
  };
}
```

```

        updated_at: "2026-03-16T10:05:00Z",
    },
    {
        id: "ROOM_102",
        room_number: "102",
        room_name: "Room 102",
        room_type_id: "RT_STD",
        floor: "1",
        building: "Main",
        status: "dirty",
        active: true,
        updated_at: "2026-03-16T10:05:00Z",
    },
    {
        id: "ROOM_201",
        room_number: "201",
        room_name: "Room 201",
        room_type_id: "RT_SUP",
        floor: "2",
        building: "Main",
        status: "clean",
        active: true,
        updated_at: "2026-03-16T10:05:00Z",
    },
],
nextCursor: null,
sourceUpdatedAtMax: "2026-03-16T10:05:00Z",
};
}
export function getMockTesiproGuests(): PmsPagedResult<Record<string, unknown>>
{
    return {
        items: [
            {
                guest_id: "GST_001",
                profile_id: "PROF_001",
                first_name: "Juan",
                last_name: "Pérez",
                full_name: "Juan Pérez",
                email: "juan.perez@example.com",
                phone: "+34600000001",
                document_type: "DNI",
                document_number: "12345678A",
                country: "ESP",
                nationality: "ESP",
                birth_date: "1985-04-12",
                gender: "M",
                updated_at: "2026-03-16T10:10:00Z",
            },
            {
                guest_id: "GST_002",
                profile_id: "PROF_002",
                first_name: "María",
                last_name: "Gómez",
                full_name: "María Gómez",
                email: "maria.gomez@example.com",
                phone: "+34600000002",
                document_type: "PASSPORT",
                document_number: "XK998877",
                country: "ESP",
                nationality: "ESP",
                birth_date: "1990-09-03",
                gender: "F",
                updated_at: "2026-03-16T10:10:00Z",
            },
        ],
    };
}

```

```

    },
  ],
  nextCursor: null,
  sourceUpdatedAtMax: "2026-03-16T10:10:00Z",
};
}
export function getMockTesiproReservations(): PmsPagedResult<Record<string,
unknown>> {
  return {
    items: [
      {
        booking_id: "RSV_1001",
        booking_status: "confirmed",
        guest_id: "GST_001",
        client_name: "Juan Pérez",
        arrival_date: "2026-03-20",
        departure_date: "2026-03-23",
        created_at: "2026-03-10T09:30:00Z",
        updated_at: "2026-03-16T10:15:00Z",
        room_type_id: "RT_STD",
        room_id: "ROOM_101",
        adults: 2,
        children: 0,
        total_amount: 360.5,
        currency: "EUR",
        channel: "Booking.com",
        segment: "OTA",
        rate_plan: "BAR",
      },
      {
        booking_id: "RSV_1002",
        booking_status: "inhouse",
        guest_id: "GST_002",
        client_name: "María Gómez",
        arrival_date: "2026-03-15",
        departure_date: "2026-03-18",
        created_at: "2026-03-01T11:00:00Z",
        updated_at: "2026-03-16T10:15:00Z",
        room_type_id: "RT_SUP",
        room_id: "ROOM_201",
        adults: 2,
        children: 1,
        total_amount: 525,
        currency: "EUR",
        channel: "Direct",
        segment: "LEISURE",
        rate_plan: "FLEX",
      },
    ],
    nextCursor: null,
    sourceUpdatedAtMax: "2026-03-16T10:15:00Z",
  };
}
export function getMockTesiproStays(): PmsPagedResult<Record<string, unknown>> {
  return {
    items: [
      {
        stay_id: "STY_9001",
        reservation_id: "RSV_1001",
        guest_id: "GST_001",
        room_id: "ROOM_101",
        room_type_id: "RT_STD",
        status: "expected",
        arrival_scheduled_at: "2026-03-20T15:00:00Z",

```

```

    departure_scheduled_at: "2026-03-23T12:00:00Z",
    check_in_at: null,
    check_out_at: null,
    adults: 2,
    children: 0,
    total_amount: 360.5,
    room_revenue_amount: 320.0,
    currency: "EUR",
    updated_at: "2026-03-16T10:20:00Z",
  },
  {
    stay_id: "STY_9002",
    reservation_id: "RSV_1002",
    guest_id: "GST_002",
    room_id: "ROOM_201",
    room_type_id: "RT_SUP",
    status: "inhouse",
    arrival_scheduled_at: "2026-03-15T15:00:00Z",
    departure_scheduled_at: "2026-03-18T12:00:00Z",
    check_in_at: "2026-03-15T16:12:00Z",
    check_out_at: null,
    adults: 2,
    children: 1,
    total_amount: 525.0,
    room_revenue_amount: 470.0,
    currency: "EUR",
    updated_at: "2026-03-16T10:20:00Z",
  },
],
nextCursor: null,
sourceUpdatedAtMax: "2026-03-16T10:20:00Z",
};
}

```

2) helper para activar mock mode

supabase/functions/_shared/pms/utils/mockMode.ts

```

// supabase/functions/_shared/pms/utils/mockMode.ts
export function isPmsMockModeEnabled(providerCode?: string): boolean {
  const mockMode = (Deno.env.get("PMS MOCK_MODE") ?? "").trim().toLowerCase();
  const mockProvider = (Deno.env.get("PMS MOCK_PROVIDER") ??
    "").trim().toUpperCase();
  if (mockMode !== "true") return false;
  if (!providerCode) return true;
  if (!mockProvider) return true;
  return mockProvider === providerCode.toUpperCase();
}

```

3) actualizar TesiproConnector.ts

Aquí está la versión buena: si mock mode está activo, devuelve datos simulados; si no, falla de forma explícita porque la API real aún no está implementada.

```

// supabase/functions/_shared/pms/connectors/TesiproConnector.ts
import { PMSBaseConnector } from "../PMSBaseConnector.ts";
import { PmsPagedResult, PmsSyncContext } from "../types/pms.types.ts";
import {
  getMockTesiproGuests,
  getMockTesiproReservations,
  getMockTesiproRooms,
  getMockTesiproRoomTypes,
  getMockTesiproStays,
} from "../mocks/tesipro.mock.ts";
import { isPmsMockModeEnabled } from "../utils/mockMode.ts";
export class TesiproConnector extends PMSBaseConnector {
  providerCode = "TESIPRO_ULYSES";
  private getBaseUrl(_ctx: PmsSyncContext): string {
    const envUrl = Deno.env.get("TESIPRO_API_BASE_URL");
    if (!envUrl) {
      throw new Error("TESIPRO_API_BASE_URL is not configured");
    }
    return envUrl;
  }
  private ensureRealModeReady(ctx: PmsSyncContext) {
    if (isPmsMockModeEnabled(this.providerCode)) return;
    this.getBaseUrl(ctx);
  }
  async fetchRoomTypes(
    ctx: PmsSyncContext,
  ): Promise<PmsPagedResult<Record<string, unknown>>> {
    if (isPmsMockModeEnabled(this.providerCode)) {
      return getMockTesiproRoomTypes();
    }
    this.ensureRealModeReady(ctx);
    throw new Error("Tesipro fetchRoomTypes pending API validation");
  }
  async fetchRooms(
    ctx: PmsSyncContext,
  ): Promise<PmsPagedResult<Record<string, unknown>>> {
    if (isPmsMockModeEnabled(this.providerCode)) {
      return getMockTesiproRooms();
    }
    this.ensureRealModeReady(ctx);
    throw new Error("Tesipro fetchRooms pending API validation");
  }
  async fetchGuests(
    ctx: PmsSyncContext,
  ): Promise<PmsPagedResult<Record<string, unknown>>> {
    if (isPmsMockModeEnabled(this.providerCode)) {
      return getMockTesiproGuests();
    }
    this.ensureRealModeReady(ctx);
    throw new Error("Tesipro fetchGuests pending API validation");
  }
  async fetchReservations(
    ctx: PmsSyncContext,
  ): Promise<PmsPagedResult<Record<string, unknown>>> {
    if (isPmsMockModeEnabled(this.providerCode)) {
      return getMockTesiproReservations();
    }
    this.ensureRealModeReady(ctx);
    throw new Error("Tesipro fetchReservations pending API validation");
  }
  async fetchStays(
    ctx: PmsSyncContext,
  ): Promise<PmsPagedResult<Record<string, unknown>>> {
    if (isPmsMockModeEnabled(this.providerCode)) {

```

```
        return getMockTesiproStays();
    }
    this.ensureRealModeReady(ctx);
    throw new Error("Tesipro fetchStays pending API validation");
}
}
```

4) ajuste necesario en `tesipro.types.ts`

El mock de reservas usa `guest_id`, así que añádelo en el tipo `raw` de reserva:

```
export interface TesiproReservationRaw {
    booking_id?: string;
    reservation_id?: string;
    booking_status?: string;
    status?: string;
    guest_id?: string;
    client_name?: string;
    guest_name?: string;
    arrival_date?: string;
    departure_date?: string;
    created_at?: string;
    updated_at?: string;
    email?: string;
    phone?: string;
    room_type_id?: string;
    room_id?: string;
    adults?: number | string;
    children?: number | string;
    total_amount?: number | string;
    currency?: string;
    channel?: string;
    segment?: string;
    rate_plan?: string;
}
```

5) ajuste necesario en `tesipro.mapper.ts`

Tu mapper de reservas debe mapear `guest_id` a `external_guest_id`. Cambia esa parte:

```
export function mapTesiproReservation(
    raw: TesiproReservationRaw,
    base: { orgId: string; propertyId: string; connectionId: string },
): CanonicalReservation {
    const rawStatus = toNullableString(raw.booking_status ?? raw.status);
    return {
        org_id: base.orgId,
        property_id: base.propertyId,
        connection_id: base.connectionId,
        external_reservation_id:
            toNullableString(raw.booking_id ?? raw.reservation_id) ?? "",
        external_confirmation_code: null,
        external_group_reservation_id: null,
    };
}
```

```

external_guest_id: toNullableString(raw.guest_id),
external_primary_room_type_id: toNullableString(raw.room_type_id),
external_assigned_room_id: toNullableString(raw.room_id),
status: mapReservationStatus(rawStatus),
raw_status: rawStatus,
booked_at: toNullableString(raw.created_at),
check_in_date: toNullableString(raw.arrival_date),
check_out_date: toNullableString(raw.departure_date),
nights: null,
adults: toNullableNumber(raw.adults),
children: toNullableNumber(raw.children),
infants: null,
rooms_count: 1,
channel_code: null,
channel_name: toNullableString(raw.channel),
segment_code: null,
segment_name: toNullableString(raw.segment),
rate_plan_code: null,
rate_plan_name: toNullableString(raw.rate_plan),
currency_code: toNullableString(raw.currency),
room_revenue_amount: null,
total_amount: toNullableNumber(raw.total_amount),
paid_amount: null,
balance_amount: null,
cancellation_at: null,
source_updated_at: toNullableString(raw.updated_at),
};
}

```

6) variables de entorno

Para probar el mock, define esto en Supabase Functions secrets o en local:

PMS MOCK_MODE=true

PMS MOCK_PROVIDER=TESIPRO_ULYSES

No necesitas TESIPRO_API_BASE_URL para el mock.

7) orden correcto de prueba

Ahora sí puedes probar el pipeline completo.

primero

ROOM_TYPE

```

{
  "connectionId": "TU_CONNECTION_ID",
  "entityType": "ROOM_TYPE",
  "syncMode": "FULL",
  "propertyId": "TU_PROPERTY_ID",
  "limit": 100
}

```

segundo

ROOM

```
{
  "connectionId": "TU_CONNECTION_ID",
  "entityType": "ROOM",
  "syncMode": "FULL",
  "propertyId": "TU_PROPERTY_ID",
  "limit": 100
}
```

tercero

GUEST

```
{
  "connectionId": "TU_CONNECTION_ID",
  "entityType": "GUEST",
  "syncMode": "FULL",
  "propertyId": "TU_PROPERTY_ID",
  "limit": 100
}
```

cuarto

RESERVATION

```
{
  "connectionId": "TU_CONNECTION_ID",
  "entityType": "RESERVATION",
  "syncMode": "FULL",
  "propertyId": "TU_PROPERTY_ID",
  "limit": 100
}
```

quinto

STAY

```
{
  "connectionId": "TU_CONNECTION_ID",
  "entityType": "STAY",
  "syncMode": "FULL",
  "propertyId": "TU_PROPERTY_ID",
  "limit": 100
}
```

8) qué deberías ver en base

pms_sync_jobs

5 jobs con SUCCESS

pms_raw_records

registros raw insertados para cada entidad

pms_room_types

3 room types

pms_rooms

3 habitaciones

pms_guests

2 huéspedes

pms_reservations

2 reservas

pms_stays

2 estancias

9) SQL de comprobación útil

room types

```
select
  external_room_type_id,
  code,
  name,
  max_occupancy,
  is_active
from pms_room_types
order by name;
```

rooms

```
select
  external_room_id,
  room_number,
  room_name,
  room_type_id,
  operational_status,
  raw_status
from pms_rooms
order by room_number;
```

guests

```
select
  external_guest_id,
  full_name,
  email,
  document_number,
  country_code
```

```
from pms_guests
order by full_name;
```

reservations

```
select
  external_reservation_id,
  guest_id,
  primary_room_type_id,
  assigned_room_id,
  status,
  raw_status,
  check_in_date,
  check_out_date,
  total_amount
from pms_reservations
order by check_in_date;
```

stays

```
select
  external_stay_id,
  reservation_id,
  guest_id,
  room_id,
  room_type_id,
  stay_status,
  raw_status,
  arrival_scheduled_at,
  departure_scheduled_at
from pms_stays
order by arrival_scheduled_at;
```

10) problema real que puedes encontrar

Hay uno bastante probable:

pms_stays no tiene unique (connection_id, external_stay_id)

Tu repo hace `upsert(... onConflict: "connection_id,external_stay_id")`, pero en el SQL inicial de `pms_stays` no dejé esa restricción única.

Tienes que añadirla.

SQL

```
alter table pms_stays
add constraint pms_stays_connection_external_stay_unique
unique (connection_id, external_stay_id);
```

11) otro ajuste recomendable en `pms_raw_records`

Ahora mismo `insertMany` siempre inserta. Si repites el mismo sync muchas veces, vas a acumular raw duplicado. Eso puede ser aceptable al principio para auditoría, pero crecerá rápido.

En fase mock no pasa nada grave. En producción, conviene decidir entre:

- guardar histórico raw completo por job,
- o hacer deduplicación por `payload_hash`.

De momento, para probar, déjalo así.

12) lectura honesta

Con esto no estás integrando Tesipro de verdad todavía.

Estás haciendo algo más importante primero: **probar de extremo a extremo la arquitectura de ingestión PMS de Debacu.**

Eso es correcto.

Si la arquitectura falla ya en mock, habría sido absurdo ponerse antes con OAuth, endpoints y partner onboarding.

Vamos con el script de limpieza/reset por `connection_id`.

Esto te sirve para repetir pruebas del pipeline PMS sin ir dejando basura en:

- `pms_stays`
- `pms_reservations`
- `pms_guests`
- `pms_rooms`
- `pms_room_types`
- `pms_raw_records`
- `pms_sync_jobs`

Y, si quieres, también:

- `pms_property_mappings`

- pms_connections

Te lo separo en dos niveles:

1. **reset de datos sincronizados**, manteniendo la conexión
 2. **reset total**, borrando también mapping y conexión
-

1) RESET SEGURO

mantiene la conexión PMS

Este es el que usarás casi siempre.

Sustituye:

- YOUR_CONNECTION_ID

```
-- =====
-- RESET SEGURO PMS POR CONNECTION_ID
-- Mantiene la conexión y el property mapping
-- Borra datos sincronizados, raw records y jobs
-- =====
do $$
declare
    v_connection_id uuid := 'YOUR_CONNECTION_ID'::uuid;
begin
    -- 1. borrar folios si existieran
    delete from pms_folio_summaries
    where connection_id = v_connection_id;
    -- 2. borrar stays
    delete from pms_stays
    where connection_id = v_connection_id;
    -- 3. borrar reservation guests
    delete from pms_reservation_guests
    where reservation_id in (
        select id
        from pms_reservations
        where connection_id = v_connection_id
    );
    -- 4. borrar reservations
    delete from pms_reservations
    where connection_id = v_connection_id;
    -- 5. borrar guests
    delete from pms_guests
    where connection_id = v_connection_id;
    -- 6. borrar rooms
    delete from pms_rooms
    where connection_id = v_connection_id;
    -- 7. borrar room types
    delete from pms_room_types
    where connection_id = v_connection_id;
    -- 8. borrar raw records
    delete from pms_raw_records
    where connection_id = v_connection_id;
    -- 9. borrar jobs
    delete from pms_sync_jobs
```

```

where connection_id = v_connection_id;
-- 10. reset estado de conexión
update pms_connections
set
    status = 'PENDING',
    last_sync_at = null,
    last_success_sync_at = null,
    last_error_at = null,
    last_error_message = null,
    updated_at = now()
where id = v_connection_id;
raise notice 'Reset seguro completado para connection_id=%', v_connection_id;
end $$;

```

2) RESET TOTAL

borra también mapping y conexión

Esto ya es para dejarla totalmente limpia.

```

-- =====
-- RESET TOTAL PMS POR CONNECTION_ID
-- Borra datos sincronizados, jobs, raw, mappings y conexión
-- =====
do $$
declare
    v_connection_id uuid := 'YOUR_CONNECTION_ID'::uuid;
begin
    -- 1. borrar folios
    delete from pms_folio_summaries
    where connection_id = v_connection_id;
    -- 2. borrar stays
    delete from pms_stays
    where connection_id = v_connection_id;
    -- 3. borrar reservation guests
    delete from pms_reservation_guests
    where reservation_id in (
        select id
        from pms_reservations
        where connection_id = v_connection_id
    );
    -- 4. borrar reservations
    delete from pms_reservations
    where connection_id = v_connection_id;
    -- 5. borrar guests
    delete from pms_guests
    where connection_id = v_connection_id;
    -- 6. borrar rooms
    delete from pms_rooms
    where connection_id = v_connection_id;
    -- 7. borrar room types
    delete from pms_room_types
    where connection_id = v_connection_id;
    -- 8. borrar raw records
    delete from pms_raw_records
    where connection_id = v_connection_id;
    -- 9. borrar jobs
    delete from pms_sync_jobs
    where connection_id = v_connection_id;

```

```
-- 10. borrar property mappings
delete from pms_property_mappings
where connection_id = v_connection_id;
-- 11. borrar conexión
delete from pms_connections
where id = v_connection_id;
raise notice 'Reset total completado para connection_id=%', v_connection_id;
end $$;
```

3) RESET POR org_id + property_id + provider

Esto es útil si no quieres andar copiando el connection_id.

Sustituye:

- YOUR_ORG_ID
- YOUR_PROPERTY_ID

```
do $$
declare
  v_org_id uuid := 'YOUR_ORG_ID'::uuid;
  v_property_id uuid := 'YOUR_PROPERTY_ID'::uuid;
  v_connection_id uuid;
begin
  select id
  into v_connection_id
  from pms_connections
  where org_id = v_org_id
    and property_id = v_property_id
    and provider_code = 'TESIPRO_ULYSES'
  order by created_at desc
  limit 1;
  if v_connection_id is null then
    raise exception 'No se encontró connection para org=% property=%
provider=TESIPRO_ULYSES',
      v_org_id, v_property_id;
  end if;
  delete from pms_folio_summaries where connection_id = v_connection_id;
  delete from pms_stays where connection_id = v_connection_id;
  delete from pms_reservation_guests
  where reservation_id in (
    select id from pms_reservations where connection_id = v_connection_id
  );
  delete from pms_reservations where connection_id = v_connection_id;
  delete from pms_guests where connection_id = v_connection_id;
  delete from pms_rooms where connection_id = v_connection_id;
  delete from pms_room_types where connection_id = v_connection_id;
  delete from pms_raw_records where connection_id = v_connection_id;
  delete from pms_sync_jobs where connection_id = v_connection_id;
  update pms_connections
  set
    status = 'PENDING',
    last_sync_at = null,
    last_success_sync_at = null,
    last_error_at = null,
    last_error_message = null,
    updated_at = now();
end;
```

```
where id = v_connection_id;
raise notice 'Reset seguro completado para connection_id=%', v_connection_id;
end $$;
```

4) SCRIPT DE COMPROBACIÓN ANTES DE BORRAR

Haz esto antes de ejecutar un reset, para ver qué vas a borrar.

```
select 'pms_folio_summaries' as table_name, count(*) as total
from pms_folio_summaries
where connection_id = 'YOUR_CONNECTION_ID'::uuid
union all
select 'pms_stays', count(*)
from pms_stays
where connection_id = 'YOUR_CONNECTION_ID'::uuid
union all
select 'pms_reservations', count(*)
from pms_reservations
where connection_id = 'YOUR_CONNECTION_ID'::uuid
union all
select 'pms_guests', count(*)
from pms_guests
where connection_id = 'YOUR_CONNECTION_ID'::uuid
union all
select 'pms_rooms', count(*)
from pms_rooms
where connection_id = 'YOUR_CONNECTION_ID'::uuid
union all
select 'pms_room_types', count(*)
from pms_room_types
where connection_id = 'YOUR_CONNECTION_ID'::uuid
union all
select 'pms_raw_records', count(*)
from pms_raw_records
where connection_id = 'YOUR_CONNECTION_ID'::uuid
union all
select 'pms_sync_jobs', count(*)
from pms_sync_jobs
where connection_id = 'YOUR_CONNECTION_ID'::uuid
union all
select 'pms_property_mappings', count(*)
from pms_property_mappings
where connection_id = 'YOUR_CONNECTION_ID'::uuid;
```

5) SCRIPT DE COMPROBACIÓN DESPUÉS DEL RESET

```
select
  c.id,
  c.provider_code,
  c.status,
  c.last_sync_at,
  c.last_success_sync_at,
  c.last_error_at,
```

```

    c.last_error_message
from pms_connections c
where c.id = 'YOUR_CONNECTION_ID'::uuid;
select 'pms_room_types' as table_name, count(*) as total
from pms_room_types
where connection_id = 'YOUR_CONNECTION_ID'::uuid
union all
select 'pms_rooms', count(*)
from pms_rooms
where connection_id = 'YOUR_CONNECTION_ID'::uuid
union all
select 'pms_guests', count(*)
from pms_guests
where connection_id = 'YOUR_CONNECTION_ID'::uuid
union all
select 'pms_reservations', count(*)
from pms_reservations
where connection_id = 'YOUR_CONNECTION_ID'::uuid
union all
select 'pms_stays', count(*)
from pms_stays
where connection_id = 'YOUR_CONNECTION_ID'::uuid
union all
select 'pms_raw_records', count(*)
from pms_raw_records
where connection_id = 'YOUR_CONNECTION_ID'::uuid
union all
select 'pms_sync_jobs', count(*)
from pms_sync_jobs
where connection_id = 'YOUR_CONNECTION_ID'::uuid;

```

6) FUNCIÓN SQL REUTILIZABLE

Si vas a repetir esto muchas veces, mejor dejar una función.

función reset seguro

```

create or replace function reset_pms_connection_data(p_connection_id uuid)
returns void
language plpgsql
as $$
begin
    delete from pms_folio_summaries
    where connection_id = p_connection_id;
    delete from pms_stays
    where connection_id = p_connection_id;
    delete from pms_reservation_guests
    where reservation_id in (
        select id
        from pms_reservations
        where connection_id = p_connection_id
    );
    delete from pms_reservations
    where connection_id = p_connection_id;
    delete from pms_guests
    where connection_id = p_connection_id;
    delete from pms_rooms
    where connection_id = p_connection_id;
    delete from pms_room_types
    where connection_id = p_connection_id;

```



```

delete from pms_raw_records
where connection_id = p_connection_id;
delete from pms_sync_jobs
where connection_id = p_connection_id;
update pms_connections
set
    status = 'PENDING',
    last_sync_at = null,
    last_success_sync_at = null,
    last_error_at = null,
    last_error_message = null,
    updated_at = now()
where id = p_connection_id;
end;
$$;

```

uso

```
select reset_pms_connection_data('YOUR_CONNECTION_ID'::uuid);
```

7) VERSIÓN MÁS SEGURA AÚN: SOLO SI EL PROVIDER ES TESIPRO

Esto evita borrar por error otra integración.

```

create or replace function reset_tesipro_connection_data(p_connection_id uuid)
returns void
language plpgsql
as $$
declare
    v_provider text;
begin
    select provider_code
    into v_provider
    from pms_connections
    where id = p_connection_id;
    if v_provider is null then
        raise exception 'Connection no encontrada: %', p_connection_id;
    end if;
    if v_provider <> 'TESIPRO_ULYSES' then
        raise exception 'La connection % no es TESIPRO_ULYSES, es %',
        p_connection_id, v_provider;
    end if;
    perform reset_pms_connection_data(p_connection_id);
end;
$$;

```

uso

```
select reset_tesipro_connection_data('YOUR_CONNECTION_ID'::uuid);
```

8) CONSEJO PRÁCTICO

Para pruebas de mock, yo usaría este ciclo siempre:

1. reset seguro

2. sync ROOM_TYPE
3. sync ROOM
4. sync GUEST
5. sync RESERVATION
6. sync STAY
7. revisar datos
8. repetir

Eso te da un circuito estable.

9) RIESGO REAL QUE DEBES VIGILAR

Hay un detalle importante: ahora mismo `pms_raw_records` se borra por `connection_id`, no por job. Eso está bien para un reset completo de pruebas, pero en producción quizá no quieras borrar auditoría raw histórica tan alegremente.

Para entorno de desarrollo y test, sí.

Para producción, convendrá separar:

- reset de entorno técnico,
- retención histórica raw,
- borrado lógico o archivado.

Con esto ya puedes iterar bien el mock.

Seguimos con una prueba antes de diseñar UI

La idea es que puedas ejecutar una sola orden y que haga esto:

1. ROOM_TYPE
2. ROOM
3. GUEST
4. RESERVATION
5. STAY

y además te muestre:

- qué llamada lanza,
- respuesta de cada sync,
- si algo falla, parar ahí.

1. Enfoque recomendado

Te propongo dos piezas:

A. Script bash simple

Útil para probar rápido desde terminal.

B. Script TypeScript/Deno

Más limpio, más mantenible y mejor para DebaCu a medio plazo.

Lo más sensato es tener **los dos**:

- bash para salir del paso,
 - TypeScript para dejarlo serio.
-

2. Script bash simple

Guárdalo como:

```
scripts/pms-sync-runner.sh
#!/usr/bin/env bash
set -euo pipefail
SUPABASE_FN_URL="${SUPABASE_FN_URL:-}"
AUTH_TOKEN="${AUTH_TOKEN:-}"
CONNECTION_ID="${CONNECTION_ID:-}"
PROPERTY_ID="${PROPERTY_ID:-}"
LIMIT="${LIMIT:-100}"
if [ -z "$SUPABASE_FN_URL" ]; then
    echo "ERROR: SUPABASE_FN_URL no definido"
    exit 1
fi
if [ -z "$AUTH_TOKEN" ]; then
    echo "ERROR: AUTH_TOKEN no definido"
    exit 1
fi
if [ -z "$CONNECTION_ID" ]; then
    echo "ERROR: CONNECTION_ID no definido"
    exit 1
fi
if [ -z "$PROPERTY_ID" ]; then
    echo "ERROR: PROPERTY_ID no definido"
    exit 1
fi
run_sync() {
    local ENTITY_TYPE="$1"
    echo ""
    echo "=====
    echo "SYNC: $ENTITY_TYPE"
    echo "=====
    curl --fail --silent --show-error \
```

```

-X POST "$SUPABASE_FN_URL/functions/v1/pms_sync_run" \
-H "Authorization: Bearer $AUTH_TOKEN" \
-H "Content-Type: application/json" \
-d "{
  \"connectionId\": \"$CONNECTION_ID\",
  \"entityType\": \"$ENTITY_TYPE\",
  \"syncMode\": \"FULL\",
  \"propertyId\": \"$PROPERTY_ID\",
  \"limit\": $LIMIT
}"
echo ""
}
run_sync "ROOM_TYPE"
run_sync "ROOM"
run_sync "GUEST"
run_sync "RESERVATION"
run_sync "STAY"
echo ""
echo "OK: pipeline PMS ejecutado completo"

```

3. Cómo ejecutarlo

```

chmod +x scripts/pms-sync-runner.sh
export SUPABASE_FN_URL="https://TU_PROJECT_REF.supabase.co"
export AUTH_TOKEN="TU_JWT"
export CONNECTION_ID="TU_CONNECTION_ID"
export PROPERTY_ID="TU_PROPERTY_ID"
export LIMIT="100"
./scripts/pms-sync-runner.sh

```

4. Versión TypeScript/Deno

Esta es mejor para dejarlo integrado con Debacu.

Guárdalo como:

```

scripts/pms_sync_runner.ts
// scripts/pms_sync_runner.ts
type EntityType = "ROOM_TYPE" | "ROOM" | "GUEST" | "RESERVATION" | "STAY";
interface SyncResponse {
  ok: boolean;
  syncJobId?: string;
  provider?: string;
  entityType?: string;
  read?: number;
  inserted?: number;
  updated?: number;
  nextCursor?: string | null;
  error?: string;
}
const SUPABASE_FN_URL = Deno.env.get("SUPABASE_FN_URL") ?? "";
const AUTH_TOKEN = Deno.env.get("AUTH_TOKEN") ?? "";
const CONNECTION_ID = Deno.env.get("CONNECTION_ID") ?? "";
const PROPERTY_ID = Deno.env.get("PROPERTY_ID") ?? "";
const LIMIT = Number(Deno.env.get("LIMIT") ?? "100");
if (!SUPABASE_FN_URL) throw new Error("Missing SUPABASE_FN_URL");
if (!AUTH_TOKEN) throw new Error("Missing AUTH_TOKEN");
if (!CONNECTION_ID) throw new Error("Missing CONNECTION_ID");

```

```

if (!PROPERTY_ID) throw new Error("Missing PROPERTY_ID");
const ENTITIES: EntityType[] = [
  "ROOM_TYPE",
  "ROOM",
  "GUEST",
  "RESERVATION",
  "STAY",
];
async function runSync(entityType: EntityType): Promise<SyncResponse> {
  const response = await fetch(`${SUPABASE_FN_URL}/functions/v1/pms_sync_run`, {
    method: "POST",
    headers: {
      Authorization: `Bearer ${AUTH_TOKEN}`,
      "Content-Type": "application/json",
    },
    body: JSON.stringify({
      connectionId: CONNECTION_ID,
      entityType,
      syncMode: "FULL",
      propertyId: PROPERTY_ID,
      limit: LIMIT,
    }),
  });
  const json = (await response.json()) as SyncResponse;
  if (!response.ok || !json.ok) {
    throw new Error(
      `Sync failed for ${entityType}: ${json.error ?? `HTTP ${
        response.status
      }`}`
    );
  }
  return json;
}
function printResult(entityType: EntityType, result: SyncResponse) {
  console.log("");
  console.log("=====");
  console.log(`SYNC OK: ${entityType}`);
  console.log("=====");
  console.log(`jobId:      ${result.syncJobId ?? "-"}`);
  console.log(`provider:    ${result.provider ?? "-"}`);
  console.log(`read:        ${result.read ?? 0}`);
  console.log(`inserted:    ${result.inserted ?? 0}`);
  console.log(`updated:     ${result.updated ?? 0}`);
  console.log(`nextCursor:  ${result.nextCursor ?? "-"}`);
}
async function main() {
  console.log("Iniciando pipeline PMS...");
  console.log(`connectionId: ${CONNECTION_ID}`);
  console.log(`propertyId:    ${PROPERTY_ID}`);
  console.log(`limit:         ${LIMIT}`);
  for (const entity of ENTITIES) {
    console.log("");
    console.log(`>>> Ejecutando ${entity}...`);
    const result = await runSync(entity);
    printResult(entity, result);
  }
  console.log("");
  console.log("Pipeline PMS completado correctamente.");
}
if (import.meta.main) {
  main().catch((error) => {
    console.error("");
    console.error("ERROR ejecutando pipeline PMS");
    console.error(error instanceof Error ? error.message : String(error));
    Deno.exit(1);
  });
}

```

```
});  
}
```

5. Cómo ejecutarlo

```
export SUPABASE_FN_URL="https://TU_PROJECT_REF.supabase.co"  
export AUTH_TOKEN="TU_JWT"  
export CONNECTION_ID="TU_CONNECTION_ID"  
export PROPERTY_ID="TU_PROPERTY_ID"  
export LIMIT="100"  
deno run --allow-env --allow-net scripts/pms_sync_runner.ts
```

6. Versión con reset previo automático

Esto ya es más útil en la práctica.

Primero resetea, luego lanza los 5 sync.

Como el reset ahora lo tienes en SQL, hay dos caminos:

camino simple

haces reset manual en SQL y luego runner.

camino mejor

creas una Edge Function interna de reset y el runner la llama primero.

Ahora mismo, como todavía no has montado la función de reset, te dejo el runner preparado para que puedas activarlo cuando exista.

Guárdalo como:

```
scripts/pms_sync_runner_with_reset.ts  
// scripts/pms_sync_runner_with_reset.ts  
type EntityType = "ROOM_TYPE" | "ROOM" | "GUEST" | "RESERVATION" | "STAY";  
const SUPABASE_FN_URL = Deno.env.get("SUPABASE_FN_URL") ?? "";  
const AUTH_TOKEN = Deno.env.get("AUTH_TOKEN") ?? "";  
const CONNECTION_ID = Deno.env.get("CONNECTION_ID") ?? "";  
const PROPERTY_ID = Deno.env.get("PROPERTY_ID") ?? "";  
const LIMIT = Number(Deno.env.get("LIMIT") ?? "100");  
const ENABLE_RESET = (Deno.env.get("ENABLE_RESET") ?? "false").toLowerCase() ===  
"true";  
if (!SUPABASE_FN_URL) throw new Error("Missing SUPABASE_FN_URL");  
if (!AUTH_TOKEN) throw new Error("Missing AUTH_TOKEN");  
if (!CONNECTION_ID) throw new Error("Missing CONNECTION_ID");  
if (!PROPERTY_ID) throw new Error("Missing PROPERTY_ID");  
const ENTITIES: EntityType[] = ["ROOM_TYPE", "ROOM", "GUEST", "RESERVATION",  
"STAY"];  
async function callFunction(path: string, body: Record<string, unknown>) {  
  const response = await fetch(`${SUPABASE_FN_URL}/functions/v1/${path}`, {  
    method: "POST",  
    headers: {  
      Authorization: `Bearer ${AUTH_TOKEN}`,  
      "Content-Type": "application/json",  
    },  
  },
```

```

    body: JSON.stringify(body),
  });
  const json = await response.json();
  if (!response.ok || json?.ok === false) {
    throw new Error(`${path} failed: ${json?.error ?? `HTTP $
{response.status}`}`);
  }
  return json;
}
async function maybeReset() {
  if (!ENABLE_RESET) return;
  console.log(">>> Ejecutando reset previo...");
  await callFunction("pms_reset_connection", {
    connectionId: CONNECTION_ID,
  });
  console.log("Reset completado.");
}
async function runSync(entityType: EntityType) {
  console.log(">>> Ejecutando ${entityType}...");
  const result = await callFunction("pms_sync_run", {
    connectionId: CONNECTION_ID,
    entityType,
    syncMode: "FULL",
    propertyId: PROPERTY_ID,
    limit: LIMIT,
  });
  console.log(
    `[${entityType}] job=${result.syncJobId ?? "-"} read=${result.read ?? 0}
inserted=${result.inserted ?? 0} updated=${result.updated ?? 0}`,
  );
}
async function main() {
  await maybeReset();
  for (const entity of ENTITIES) {
    await runSync(entity);
  }
  console.log("Pipeline completo OK.");
}
if (import.meta.main) {
  main().catch((error) => {
    console.error(error instanceof Error ? error.message : String(error));
    Deno.exit(1);
  });
}

```

7. Recomendación práctica de uso

Para desarrollo yo usaría este ciclo:

paso 1

reset SQL manual

paso 2

runner TypeScript

deno run --allow-env --allow-net scripts/pms_sync_runner.ts

paso 3

consultas SQL de verificación

8. SQL de verificación rápida tras el runner

```
select entity_type, status, records_read, records_inserted, records_failed
from pms_sync_jobs
where connection_id = 'YOUR_CONNECTION_ID'::uuid
order by started_at desc;
select count(*) as room_types from pms_room_types where connection_id =
'YOUR_CONNECTION_ID'::uuid;
select count(*) as rooms      from pms_rooms where connection_id =
'YOUR_CONNECTION_ID'::uuid;
select count(*) as guests      from pms_guests where connection_id =
'YOUR_CONNECTION_ID'::uuid;
select count(*) as reservations from pms_reservations where connection_id =
'YOUR_CONNECTION_ID'::uuid;
select count(*) as stays        from pms_stays where connection_id =
'YOUR_CONNECTION_ID'::uuid;
```

Con el mock actual deberías ver:

- room_types: 3
 - rooms: 3
 - guests: 2
 - reservations: 2
 - stays: 2
-

9. Archivo .env de ejemplo para local

Puedes tener algo así en local, sin subirlo al repo:

```
SUPABASE_FN_URL=https://TU_PROJECT_REF.supabase.co
AUTH_TOKEN=TU_JWT
CONNECTION_ID=TU_CONNECTION_ID
PROPERTY_ID=TU_PROPERTY_ID
LIMIT=100
```

Y luego cargarlo como prefieras.

10. Ajuste recomendable

Ahora mismo el runner asume syncMode: "FULL" siempre.

Para más adelante, conviene aceptar por variable de entorno:

SYNC_MODE=FULL

0

SYNC_MODE=INCREMENTAL

No es urgente, pero sí conveniente.

A: Edge Function `pms_reset_connection`, y después **B: base de pantalla interna** React/TypeScript para administrar conexiones PMS, lanzar sync y ver jobs/errores.

A) EDGE FUNCTION

`pms_reset_connection`

Objetivo

Poder resetear una conexión PMS desde una función interna, sin entrar a SQL manual cada vez.

Debe:

- recibir `connectionId`
 - validar usuario autenticado
 - borrar datos sincronizados de esa conexión
 - mantener la conexión
 - dejar la conexión en estado PENDING
-

1. Estructura

```
supabase/functions/  
  pms_reset_connection/  
    index.ts  
  _shared/pms/  
    services/  
      PmsResetService.ts
```

2. `PmsResetService.ts`

```
// supabase/functions/_shared/pms/services/PmsResetService.ts  
import { getServiceClient } from "../../db.ts";  
import { PmsConnectionRepository } from  
  "../../repositories/PmsConnectionRepository.ts";  
export class PmsResetService {  
  private db = getServiceClient();  
  private connections = new PmsConnectionRepository();  
  async resetConnectionData(connectionId: string) {  
    const connection = await this.connections.getById(connectionId);  
    // 1. folios
```

```

{
  const { error } = await this.db
    .from("pms_folio_summaries")
    .delete()
    .eq("connection_id", connectionId);
  if (error) throw error;
}
// 2. stays
{
  const { error } = await this.db
    .from("pms_stays")
    .delete()
    .eq("connection_id", connectionId);
  if (error) throw error;
}
// 3. reservation guests
{
  const { data: reservations, error: reservationsError } = await this.db
    .from("pms_reservations")
    .select("id")
    .eq("connection_id", connectionId);
  if (reservationsError) throw reservationsError;
  const reservationIds = (reservations ?? []).map((r) => r.id as string);
  if (reservationIds.length > 0) {
    const { error } = await this.db
      .from("pms_reservation_guests")
      .delete()
      .in("reservation_id", reservationIds);
    if (error) throw error;
  }
}
// 4. reservations
{
  const { error } = await this.db
    .from("pms_reservations")
    .delete()
    .eq("connection_id", connectionId);
  if (error) throw error;
}
// 5. guests
{
  const { error } = await this.db
    .from("pms_guests")
    .delete()
    .eq("connection_id", connectionId);
  if (error) throw error;
}
// 6. rooms
{
  const { error } = await this.db
    .from("pms_rooms")
    .delete()
    .eq("connection_id", connectionId);
  if (error) throw error;
}
// 7. room types
{
  const { error } = await this.db
    .from("pms_room_types")
    .delete()
    .eq("connection_id", connectionId);
  if (error) throw error;
}
// 8. raw records

```

```

{
  const { error } = await this.db
    .from("pms_raw_records")
    .delete()
    .eq("connection_id", connectionId);
  if (error) throw error;
}
// 9. sync jobs
{
  const { error } = await this.db
    .from("pms_sync_jobs")
    .delete()
    .eq("connection_id", connectionId);
  if (error) throw error;
}
// 10. reset estado conexión
{
  const { error } = await this.db
    .from("pms_connections")
    .update({
      status: "PENDING",
      last_sync_at: null,
      last_success_sync_at: null,
      last_error_at: null,
      last_error_message: null,
      updated_at: new Date().toISOString(),
    })
    .eq("id", connectionId);
  if (error) throw error;
}
return {
  ok: true,
  connectionId,
  providerCode: connection.provider_code,
};
}
}

```

3. pms_reset_connection/index.ts

```

// supabase/functions/pms_reset_connection/index.ts
import "jsr:@supabase/functions-js/edge-runtime.d.ts";
import { preflight, json } from "../_shared/cors.ts";
import { requireUser } from "../_shared/auth.ts";
import { PmsResetService } from "../_shared/pms/services/PmsResetService.ts";
type ResetInput = {
  connectionId?: string;
};
Deno.serve(async (req) => {
  if (req.method === "OPTIONS") return preflight(req);
  try {
    await requireUser(req);
    if (req.method !== "POST") {
      return json({ ok: false, error: "method_not_allowed" }, { status: 405 });
    }
    const body = (await req.json()) as ResetInput;
    if (!body?.connectionId) {
      return json({ ok: false, error: "connectionId_required" }, { status:
400 });
    }
    const service = new PmsResetService();

```

```
    const result = await service.resetConnectionData(body.connectionId);
    return json(result, { status: 200 });
  } catch (error) {
    const message = error instanceof Error ? error.message : "unexpected_error";
    return json({ ok: false, error: message }, { status: 500 });
  }
});
```

4. desplegar

supabase functions deploy pms_reset_connection

Si pruebas en local:

supabase functions serve --env-file .env.local

5. prueba rápida con curl

```
curl -X POST
"https://TU_PROJECT_REF.supabase.co/functions/v1/pms_reset_connection" \
-H "Authorization: Bearer TU_JWT" \
-H "Content-Type: application/json" \
-d '{
  "connectionId": "TU_CONNECTION_ID"
}'
```

6. ya puedes usar el runner con reset

export ENABLE_RESET=true

deno run --allow-env --allow-net scripts/pms_sync_runner_with_reset.ts

B) PANTALLA INTERNA REACT/TS DE ADMINISTRACIÓN PMS

Objetivo

Una pantalla interna, simple y útil, para:

- listar conexiones PMS
- lanzar reset
- lanzar sync manual por entidad
- ver últimos jobs
- ver errores de conexión

No hace falta hacerla bonita ahora. Hace falta que sirva.

1. qué mostraría

bloque 1: conexiones PMS

Tabla con:

- provider
- org_id
- property_id
- environment
- auth_mode
- status
- last_sync_at
- last_success_sync_at
- last_error_message

bloque 2: acciones

Por cada conexión:

- botón Reset
- botón Sync ROOM_TYPE
- botón Sync ROOM
- botón Sync GUEST
- botón Sync RESERVATION
- botón Sync STAY

bloque 3: jobs recientes

Tabla con:

- started_at
- entity_type
- sync_mode
- status
- records_read
- records_inserted
- records_failed

- error_summary
-

2. servicios frontend

Te propongo algo muy simple.

src/services/pmsAdmin.service.ts

```
// src/services/pmsAdmin.service.ts
import { supabase } from "@lib/supabaseClient";
export type PmsConnectionRow = {
  id: string;
  org_id: string;
  property_id: string | null;
  provider_code: string;
  environment: "sandbox" | "production";
  auth_mode: string;
  status: string;
  last_sync_at: string | null;
  last_success_sync_at: string | null;
  last_error_at: string | null;
  last_error_message: string | null;
  created_at: string;
  updated_at: string;
};
export type PmsSyncJobRow = {
  id: string;
  connection_id: string;
  provider_code: string;
  entity_type: string;
  sync_mode: string;
  status: string;
  records_read: number;
  records_inserted: number;
  records_updated: number;
  records_failed: number;
  error_summary: string | null;
  started_at: string;
  finished_at: string | null;
};
export async function listPmsConnections(): Promise<PmsConnectionRow[]> {
  const { data, error } = await supabase
    .from("pms_connections")
    .select("*")
    .order("created_at", { ascending: false });
  if (error) throw error;
  return (data ?? []) as PmsConnectionRow[];
}
export async function listPmsJobs(connectionId?: string):
Promise<PmsSyncJobRow[]> {
  let query = supabase
    .from("pms_sync_jobs")
    .select("*")
    .order("started_at", { ascending: false })
    .limit(50);
  if (connectionId) {
    query = query.eq("connection_id", connectionId);
  }
  const { data, error } = await query;
  if (error) throw error;
```

```

    return (data ?? []) as PmsSyncJobRow[];
  }
  export async function runPmsSync(input: {
    connectionId: string;
    entityType: "ROOM_TYPE" | "ROOM" | "GUEST" | "RESERVATION" | "STAY";
    syncMode?: "FULL" | "INCREMENTAL";
    propertyId?: string | null;
    limit?: number;
  }) {
    const { data, error } = await supabase.functions.invoke("pms_sync_run", {
      body: {
        connectionId: input.connectionId,
        entityType: input.entityType,
        syncMode: input.syncMode ?? "FULL",
        propertyId: input.propertyId ?? null,
        limit: input.limit ?? 100,
      },
    });
    if (error) throw error;
    return data;
  }
  export async function resetPmsConnection(connectionId: string) {
    const { data, error } = await
    supabase.functions.invoke("pms_reset_connection", {
      body: { connectionId },
    });
    if (error) throw error;
    return data;
  }
}

```

3. componente principal

src/pages/PmsAdminPage.tsx

```

import React, { useEffect, useMemo, useState } from "react";
import {
  listPmsConnections,
  listPmsJobs,
  resetPmsConnection,
  runPmsSync,
  type PmsConnectionRow,
  type PmsSyncJobRow,
} from "@services/pmsAdmin.service";
type SyncEntity = "ROOM_TYPE" | "ROOM" | "GUEST" | "RESERVATION" | "STAY";
const ENTITIES: SyncEntity[] = ["ROOM_TYPE", "ROOM", "GUEST", "RESERVATION", "STAY"];
export default function PmsAdminPage() {
  const [connections, setConnections] = useState<PmsConnectionRow[]>([]);
  const [jobs, setJobs] = useState<PmsSyncJobRow[]>([]);
  const [selectedConnectionId, setSelectedConnectionId] = useState<string>("");
  const [loading, setLoading] = useState(false);
  const [actionLoading, setActionLoading] = useState<string>("");
  const [message, setMessage] = useState<string>("");
  async function loadData(connectionId?: string) {
    setLoading(true);
    setMessage("");
    try {
      const [connectionsData, jobsData] = await Promise.all([
        listPmsConnections(),
        listPmsJobs(connectionId),
      ]);
    }
  }
}

```

```

        setConnections(connectionsData);
        setJobs(jobsData);
    } catch (error) {
        setMessage(error instanceof Error ? error.message : "Error loading PMS
data");
    } finally {
        setLoading(false);
    }
}
useEffect(() => {
    void loadData();
}, []);
const selectedConnection = useMemo(
    () => connections.find((c) => c.id === selectedConnectionId) ?? null,
    [connections, selectedConnectionId],
);
async function handleReset(connection: PmsConnectionRow) {
    const key = `reset:${connection.id}`;
    setActionLoading(key);
    setMessage("");
    try {
        await resetPmsConnection(connection.id);
        setMessage(`Reset OK: ${connection.provider_code}`);
        await loadData(selectedConnectionId || connection.id);
    } catch (error) {
        setMessage(error instanceof Error ? error.message : "Reset error");
    } finally {
        setActionLoading("");
    }
}
async function handleSync(connection: PmsConnectionRow, entity: SyncEntity) {
    const key = `sync:${connection.id}:${entity}`;
    setActionLoading(key);
    setMessage("");
    try {
        const result = await runPmsSync({
            connectionId: connection.id,
            entityType: entity,
            syncMode: "FULL",
            propertyId: connection.property_id,
            limit: 100,
        });
        setMessage(
            `Sync OK ${entity} | read=${result?.read ?? 0} inserted=${
{result?.inserted ?? 0}`,
        );
        await loadData(selectedConnectionId || connection.id);
    } catch (error) {
        setMessage(error instanceof Error ? error.message : "Sync error");
    } finally {
        setActionLoading("");
    }
}
}
return (
    <div className="p-4 space-y-6">
        <div>
            <h1 className="text-xl font-semibold">PMS Admin</h1>
            <p className="text-sm text-gray-600">
                Conexiones PMS, sincronizaciones y errores.
            </p>
        </div>
        {message ? (
            <div className="rounded border border-gray-300 bg-gray-50 px-3 py-2
text-sm">

```



```

        {message}
      </div>
    ) : null}
  <section className="space-y-2">
    <div className="flex items-center gap-3">
      <label className="text-sm font-medium">Filtrar jobs por
conexión</label>
      <select
        className="border rounded px-2 py-1 text-sm"
        value={selectedConnectionId}
        onChange={(e) => {
          const value = e.target.value;
          setSelectedConnectionId(value);
          void loadData(value || undefined);
        }}
      >
        <option value="">Todas</option>
        {connections.map((connection) => (
          <option key={connection.id} value={connection.id}>
            {connection.provider_code} | {connection.property_id ?? "no-
property"} | {" "}
            {connection.status}
          </option>
        ))}
      </select>
    </div>
  </section>
  <section className="space-y-2">
    <h2 className="text-lg font-medium">Conexiones PMS</h2>
    <div className="overflow-auto border rounded">
      <table className="min-w-full text-sm">
        <thead className="bg-gray-100">
          <tr>
            <th className="text-left px-2 py-2">Provider</th>
            <th className="text-left px-2 py-2">Org</th>
            <th className="text-left px-2 py-2">Property</th>
            <th className="text-left px-2 py-2">Env</th>
            <th className="text-left px-2 py-2">Auth</th>
            <th className="text-left px-2 py-2">Status</th>
            <th className="text-left px-2 py-2">Last Sync</th>
            <th className="text-left px-2 py-2">Last Success</th>
            <th className="text-left px-2 py-2">Last Error</th>
            <th className="text-left px-2 py-2">Actions</th>
          </tr>
        </thead>
        <tbody>
          {connections.map((connection) => (
            <tr key={connection.id} className="border-t align-top">
              <td className="px-2 py-2">{connection.provider_code}</td>
              <td className="px-2 py-2">{connection.org_id}</td>
              <td className="px-2 py-2">{connection.property_id ?? "-"}</td>
              <td className="px-2 py-2">{connection.environment}</td>
              <td className="px-2 py-2">{connection.auth_mode}</td>
              <td className="px-2 py-2">{connection.status}</td>
              <td className="px-2 py-2">{connection.last_sync_at ??
"-"}</td>
              <td className="px-2 py-2">{connection.last_success_sync_at ??
"-"}</td>
              <td className="px-2 py-2 max-w-[260px] whitespace-pre-wrap">
                {connection.last_error_message ?? "-"}
              </td>
              <td className="px-2 py-2">
                <div className="flex flex-wrap gap-2">
                  <button

```

```

        className="border rounded px-2 py-1"
        disabled={actionLoading === `reset:${connection.id}`}
        onClick={() => void handleReset(connection)}
      >
        Reset
      </button>
      {ENTITIES.map((entity) => (
        <button
          key={entity}
          className="border rounded px-2 py-1"
          disabled={actionLoading === `sync:${connection.id}:$
{entity}`}
          onClick={() => void handleSync(connection, entity)}
        >
          Sync {entity}
        </button>
      ))}
    </div>
  </td>
</tr>
)}}
{!loading && connections.length === 0 ? (
  <tr>
    <td colSpan={10} className="px-2 py-4 text-center text-gray-
500">
      No hay conexiones PMS.
    </td>
  </tr>
) : null}
</tbody>
</table>
</div>
</section>
<section className="space-y-2">
  <h2 className="text-lg font-medium">Últimos jobs</h2>
  <div className="overflow-auto border rounded">
    <table className="min-w-full text-sm">
      <thead className="bg-gray-100">
        <tr>
          <th className="text-left px-2 py-2">Started</th>
          <th className="text-left px-2 py-2">Connection</th>
          <th className="text-left px-2 py-2">Entity</th>
          <th className="text-left px-2 py-2">Mode</th>
          <th className="text-left px-2 py-2">Status</th>
          <th className="text-left px-2 py-2">Read</th>
          <th className="text-left px-2 py-2">Inserted</th>
          <th className="text-left px-2 py-2">Updated</th>
          <th className="text-left px-2 py-2">Failed</th>
          <th className="text-left px-2 py-2">Error</th>
        </tr>
      </thead>
      <tbody>
        {jobs.map((job) => (
          <tr key={job.id} className="border-t align-top">
            <td className="px-2 py-2">{job.started_at}</td>
            <td className="px-2 py-2">{job.connection_id}</td>
            <td className="px-2 py-2">{job.entity_type}</td>
            <td className="px-2 py-2">{job.sync_mode}</td>
            <td className="px-2 py-2">{job.status}</td>
            <td className="px-2 py-2">{job.records_read}</td>
            <td className="px-2 py-2">{job.records_inserted}</td>
            <td className="px-2 py-2">{job.records_updated}</td>
            <td className="px-2 py-2">{job.records_failed}</td>
            <td className="px-2 py-2 max-w-[260px] whitespace-pre-wrap">

```

```

        {job.error_summary ?? "-"}
      </td>
    </tr>
  )}}
  {!!loading && jobs.length === 0 ? (
    <tr>
      <td colspan={10} className="px-2 py-4 text-center text-gray-
500">
        No hay jobs.
      </td>
    </tr>
  ) : null}
</tbody>
</table>
</div>
</section>
</div>
);
}

```

4. dónde meterla en navegación

Si tu app tiene una zona admin interna, la metería como vista nueva, por ejemplo:

- admin_pms
- integraciones_pms

y solo visible para admin técnico o superadmin. No la pondría accesible a usuarios hotel todavía.

5. lo que esta pantalla todavía no hace

No te vendo humo: esto es base útil, pero no está cerrada del todo.

Le falta:

- crear conexión PMS desde UI
- confirmaciones antes de reset
- selector de FULL / INCREMENTAL
- polling automático tras lanzar sync
- detalle de raw records
- filtros por provider / property
- control fino de permisos frontend

Pero como panel interno inicial, ya vale.

6. orden práctico a partir de aquí

Yo haría esto:

1. desplegar `pms_reset_connection`
2. probar runner con reset automático
3. montar esta pantalla
4. validar que:
 - reset funciona
 - sync funciona
 - jobs refrescan
 - errores se ven
5. solo después empezar con API real de Tesipro

Porque si no validas primero la operación interna, cuando metas Tesipro real no sabrás si falla la API o falla tu arquitectura.

7. siguiente paso útil

El siguiente paso más rentable ahora es prepararte el **alta de conexión PMS desde UI + Edge Function**, para no tener que crear conexiones por SQL manual.

permitir crear una conexión PMS desde la UI, sin tener que insertar filas en SQL manualmente.

La idea es:

1. **Edge Function** → crea la conexión en `pms_connections`.
2. **Formulario React** → permite crearla desde el panel admin.
3. **Validaciones mínimas** → provider, property, modo auth, entorno.
4. **Seguridad** → solo usuarios autorizados.

Esto completa el ciclo:

```
crear conexión PMS
  ↓
reset conexión
  ↓
sync manual
  ↓
ver jobs
```

1. Edge Function

pms_connection_create

Estructura

```
supabase/functions/  
  pms_connection_create/  
    index.ts
```

2.

pms_connection_create/index.ts

```
import "jsr:@supabase/functions-js/edge-runtime.d.ts";  
import { preflight, json } from "../_shared/cors.ts";  
import { requireUser, supabaseServiceClient } from "../_shared/auth.ts";  
type CreateConnectionInput = {  
  providerCode: string;  
  orgId: string;  
  propertyId?: string | null;  
  environment?: "sandbox" | "production";  
  authMode?: "api_key" | "oauth2";  
  credentials?: Record<string, unknown>;  
};  
Deno.serve(async (req) => {  
  if (req.method === "OPTIONS") return preflight(req);  
  try {  
    const user = await requireUser(req);  
    if (req.method !== "POST") {  
      return json({ ok: false, error: "method_not_allowed" }, { status: 405 });  
    }  
    const body = (await req.json()) as CreateConnectionInput;  
    if (!body.providerCode) {  
      return json({ ok: false, error: "providerCode_required" }, { status:  
400 });  
    }  
    if (!body.orgId) {  
      return json({ ok: false, error: "orgId_required" }, { status: 400 });  
    }  
    const db = supabaseServiceClient();  
    const { data, error } = await db  
      .from("pms_connections")  
      .insert({  
        provider_code: body.providerCode,  
        org_id: body.orgId,  
        property_id: body.propertyId ?? null,  
        environment: body.environment ?? "sandbox",  
        auth_mode: body.authMode ?? "api_key",  
        credentials_json: body.credentials ?? {},  
        status: "PENDING",  
      })  
      .select("*")  
      .single();  
    if (error) throw error;  
    return json(  
      {  
        ok: true,
```

```

        connection: data,
      },
      { status: 200 },
    );
  } catch (error) {
    const message = error instanceof Error ? error.message : "unexpected_error";
    return json(
      {
        ok: false,
        error: message,
      },
      { status: 500 },
    );
  }
});

```

3. desplegar

supabase functions deploy pms_connection_create

4. servicio frontend

src/services/pmsAdmin.service.ts

añade:

```

export async function createPmsConnection(input: {
  providerCode: string;
  orgId: string;
  propertyId?: string | null;
  environment?: "sandbox" | "production";
  authMode?: "api_key" | "oauth2";
  credentials?: Record<string, unknown>;
}) {
  const { data, error } = await
  supabase.functions.invoke("pms_connection_create", {
    body: input,
  });
  if (error) throw error;
  return data;
}

```

5. formulario React

PmsConnectionCreateDialog.tsx

```

import React, { useState } from "react";
import { createPmsConnection } from "@services/pmsAdmin.service";
type Props = {
  orgId: string;
  propertyId?: string | null;
  onCreate: () => void;
};
export default function PmsConnectionCreateDialog({

```

```

    orgId,
    propertyId,
    onCreated,
  }: Props) {
    const [providerCode, setProviderCode] = useState("TESIPRO_ULYSES");
    const [environment, setEnvironment] = useState<"sandbox" |
"production">("sandbox");
    const [authMode, setAuthMode] = useState<"api_key" | "oauth2">("api_key");
    const [apiKey, setApiKey] = useState("");
    const [loading, setLoading] = useState(false);
    const [message, setMessage] = useState("");
    async function handleCreate() {
      setLoading(true);
      setMessage("");
      try {
        await createPmsConnection({
          providerCode,
          orgId,
          propertyId,
          environment,
          authMode,
          credentials: {
            apiKey,
          },
        });
        setMessage("Conexión creada");
        onCreated();
      } catch (error) {
        setMessage(error instanceof Error ? error.message : "Error creando
conexión");
      } finally {
        setLoading(false);
      }
    }
    return (
      <div className="border rounded p-4 space-y-3">
        <h3 className="font-medium">Nueva conexión PMS</h3>
        <div>
          <label className="text-sm">Provider</label>
          <select
            value={providerCode}
            onChange={(e) => setProviderCode(e.target.value)}
            className="border rounded px-2 py-1 w-full"
          >
            <option value="TESIPRO_ULYSES">Tesipro Ulyses</option>
            <option value="CLOUDBEDS">Cloudbeds</option>
            <option value="MEWS">Mews</option>
            <option value="APALEO">Apaleo</option>
          </select>
        </div>
        <div>
          <label className="text-sm">Environment</label>
          <select
            value={environment}
            onChange={(e) =>
              setEnvironment(e.target.value as "sandbox" | "production")
            }
            className="border rounded px-2 py-1 w-full"
          >
            <option value="sandbox">Sandbox</option>
            <option value="production">Production</option>
          </select>
        </div>
      </div>
    );
  }
}

```

```

<label className="text-sm">Auth Mode</label>
<select
  value={authMode}
  onChange={(e) =>
    setAuthMode(e.target.value as "api_key" | "oauth2")}
  className="border rounded px-2 py-1 w-full"
>
  <option value="api_key">API Key</option>
  <option value="oauth2">OAuth2</option>
</select>
</div>
{authMode === "api_key" && (
  <div>
    <label className="text-sm">API Key</label>
    <input
      value={apiKey}
      onChange={(e) => setApiKey(e.target.value)}
      className="border rounded px-2 py-1 w-full"
    />
  </div>
)}
<button
  onClick={handleCreate}
  disabled={loading}
  className="border px-3 py-1 rounded"
>
  Crear conexión
</button>
{message && <div className="text-sm">{message}</div>}
</div>
);
}

```

6. flujo final

Ahora Debaqu PMS funciona así:

```

Admin PMS
↓
Crear conexión
↓
Reset conexión
↓
Sync ROOM_TYPE
Sync ROOM
Sync GUEST
Sync RESERVATION
Sync STAY
↓
ver jobs

```

7. algo importante que conviene añadir pronto

Antes de integrar PMS reales conviene añadir **dos campos más en pms_connections**:

rate_limit_per_minute
webhook_url
porque casi todos los PMS lo piden.

8. siguiente paso importante (muy recomendable)

Antes de conectar Tesipro real, conviene añadir **esto**:

pms_connection_test

Función que:

- prueba credenciales
- hace llamada mínima al PMS
- devuelve:

connection_valid
api_latency
permissions_ok

Esto evita errores cuando conectes PMS reales.

Si quieres, en el siguiente paso te preparo también:

la arquitectura del conector Tesipro Ulyses para Debacu (muy importante porque no funciona como Cloudbeds o Mews).

Ahí hay varios detalles técnicos que conviene decidir bien antes de empezar la integración real.

Perfecto. Vamos entonces con **la arquitectura correcta del conector Tesipro / Ulyses para Debacu**. Esto es importante hacerlo bien ahora porque **Ulyses no funciona igual que PMS como Mews, Apaleo o Cloudbeds**. Si lo estructuras mal al principio luego será difícil mantener varios PMS.

Voy a darte una **arquitectura realista que funciona para cualquier PMS**, pero optimizada para Debacu.

1. Principio clave del conector Debacu

Debacu **NO debe hablar directamente con el PMS desde toda la aplicación**.

Debacu debe usar este modelo:

Debacu Core

↓
PMS Integration Layer
↓
Provider Adapter
↓
PMS API
Es decir:

Debacu
↳ TesiproAdapter
↳ MewsAdapter
↳ ApaleoAdapter
↳ CloudbedsAdapter

Todos devuelven **el mismo modelo universal Debacu**.

2. Arquitectura interna Debacu

Dentro de Supabase Functions:

```
supabase/functions/_shared/pms/  
  adapters/  
    TesiproAdapter.ts  
    MewsAdapter.ts  
    ApaleoAdapter.ts  
    CloudbedsAdapter.ts  
  services/  
    PmsSyncService.ts  
  mappers/  
    RoomMapper.ts  
    ReservationMapper.ts  
    GuestMapper.ts  
    StayMapper.ts  
  models/  
    PmsModels.ts
```

3. Modelo universal PMS Debacu

Este es el **núcleo del sistema**.

Todos los PMS se convierten a esto.

Room Type

```
export type PmsRoomType = {  
  external_id: string  
  code?: string  
  name: string  
  description?: string  
  capacity?: number  
  created_at?: string  
  updated_at?: string  
}
```

Room

```
export type PmsRoom = {  
  external_id: string  
  room_type_external_id: string  
  number: string  
  floor?: string  
  status?: string  
}
```

Guest

```
export type PmsGuest = {  
  external_id: string  
  first_name?: string  
  last_name?: string  
  email?: string  
  phone?: string  
  document_id?: string  
  nationality?: string  
}
```

Reservation

```
export type PmsReservation = {  
  external_id: string  
  reservation_code?: string  
  status: string  
  arrival_date: string  
  departure_date: string  
  room_type_external_id?: string  
  room_external_id?: string  
  created_at?: string  
  updated_at?: string  
}
```

Stay

```
export type PmsStay = {  
  external_id: string  
  reservation_external_id: string  
  check_in?: string  
  check_out?: string  
  status: string  
}
```

4. Adapter Tesipro

TesiproAdapter.ts

```
export class TesiproAdapter {  
  async fetchRoomTypes(): Promise<PmsRoomType[]> {  
    const response = await fetch("TESIPRO_API/roomTypes")  
    const data = await response.json()  
  }  
}
```

```

    return data.map((r:any)=>({
      external_id: r.id,
      code: r.code,
      name: r.name,
      capacity: r.capacity
    })))
  }
  async fetchRooms(): Promise<PmsRoom[]> {
    const response = await fetch("TESIPRO_API/rooms")
    const data = await response.json()
    return data.map((r:any)=>({
      external_id: r.id,
      room_type_external_id: r.roomTypeId,
      number: r.number,
      floor: r.floor
    })))
  }
}

```

5. PmsSyncService

Esto es lo que llama la función `pms_sync_run`.

```

export class PmsSyncService {
  async runSync(connection, entityType) {
    const adapter = this.getAdapter(connection.provider_code)
    switch(entityType){
      case "ROOM_TYPE":
        const roomTypes = await adapter.fetchRoomTypes()
        return this.storeRoomTypes(connection, roomTypes)
      case "ROOM":
        const rooms = await adapter.fetchRooms()
        return this.storeRooms(connection, rooms)
      case "GUEST":
        const guests = await adapter.fetchGuests()
        return this.storeGuests(connection, guests)
      case "RESERVATION":
        const reservations = await adapter.fetchReservations()
        return this.storeReservations(connection, reservations)
      case "STAY":
        const stays = await adapter.fetchStays()
        return this.storeStays(connection, stays)
    }
  }
}

```

6. Selector de Adapter

```

getAdapter(providerCode){
  switch(providerCode){
    case "TESIPRO_ULYSES":
      return new TesiproAdapter()
    case "MEWS":
      return new MewsAdapter()
    case "CLOUDBEDS":
      return new CloudbedsAdapter()
    case "APALEO":
      return new ApaleoAdapter()
  }
}

```

```
        default:
            throw new Error("Unsupported PMS provider")
    }
}
```

7. Flujo real de sincronización

Cuando ejecutas:

pms_sync_run

ocurre esto:

```
pms_sync_run
  ↓
PmsSyncService
  ↓
adapter.fetchReservations()
  ↓
mapper
  ↓
insert/update DB
  ↓
pms_sync_jobs
```

8. Problema típico de PMS que hay que resolver

Cada PMS usa cosas diferentes:

PMS	guest id	reservation id	room
Mews	customerId	reservationId	spaceId
Apaleo	guestId	bookingId	unitId
Cloudbeds	guestId	reservationId	roomId
Tesipro	guestCode	bookingCode	room

Por eso necesitas **modelo universal**.

9. Estrategia correcta para Debacu

Fase 1

solo leer:

```
rooms
room types
guests
reservations
stays
```

Fase 2

añadir:

folios
charges
revenue
rate plans

10. Estrategia para el primer PMS (Tesipro)

No intentes hacer todo.

Empieza con:

ROOM_TYPE
ROOM
GUEST
RESERVATION
y **STAY** después.

11. Problema específico de Tesipro

Tesipro suele usar:

SOAP o REST híbrido
y muchas integraciones pasan por:

Channel Manager API
No siempre hay endpoint limpio para reservas.

Por eso es importante preguntarles:

¿API PMS directa?
¿API Channel Manager?
¿Webhook de reservas?

12. Lo bueno de la arquitectura Debacu

Cuando conectes otro PMS **no cambias nada del sistema.**

Solo añades:

MewsAdapter.ts
o

ApaleoAdapter.ts

13. Estructura final recomendable

_shared/pms

```
adapters/  
  TesiproAdapter.ts  
  MewsAdapter.ts  
  ApaleoAdapter.ts  
services/  
  PmsSyncService.ts  
  PmsResetService.ts  
repositories/  
  PmsConnectionRepository.ts  
  PmsSyncJobRepository.ts  
mappers/  
  ReservationMapper.ts  
  GuestMapper.ts  
models/  
  PmsModels.ts
```

14. Algo MUY importante para Debacu

Debacu no solo es PMS connector.

Debacu también hará:

CSV imports
OTA data
Revenue Intelligence
Risk evaluation

Por eso el modelo PMS **no debe depender del PMS**.

15. Siguiente paso recomendable

El siguiente paso importante ahora sería diseñar:

tabla pms_sync_jobs correcta

porque ahí vas a auditar todas las integraciones.

y te diré algo:

esa tabla mal diseñada rompe todos los conectores PMS.

Si quieres, en el siguiente paso te preparo:

la arquitectura profesional de pms_sync_jobs + pms_raw_records + sync_cursor que usan las plataformas SaaS reales (tipo Stripe / Segment / Airbyte).

Eso te ahorrará **meses de problemas cuando conectes varios PMS**.

erfecto. Vamos a diseñar **bien** las tablas críticas del sistema de sincronización PMS de Debacu. Si esto se hace mal al principio, luego aparecen problemas de duplicados, reintentos, sincronizaciones rotas o imposibilidad de depurar integraciones.

Las tres piezas fundamentales son:

1. `pms_sync_jobs` → auditoría de cada ejecución de sync
2. `pms_raw_records` → copia exacta de lo que envía el PMS
3. `pms_sync_cursors` → control incremental de sincronización

Este diseño está inspirado en arquitecturas de integración reales (Stripe, Airbyte, Segment).

1. Tabla `pms_sync_jobs`

Esta tabla registra **cada ejecución de sincronización**.

Objetivo

Saber siempre:

- qué se ejecutó
 - cuándo
 - cuántos registros llegaron
 - cuántos se insertaron
 - cuántos fallaron
 - qué error ocurrió
-

SQL

```
create table pms_sync_jobs (  
  id uuid primary key default gen_random_uuid(),  
  connection_id uuid not null references pms_connections(id),  
  provider_code text not null,  
  entity_type text not null,  
  -- ROOM_TYPE  
  -- ROOM  
  -- GUEST  
  -- RESERVATION  
  -- STAY  
  sync_mode text not null,  
  -- FULL  
  -- INCREMENTAL  
  status text not null default 'RUNNING',  
  -- RUNNING  
  -- SUCCESS  
  -- PARTIAL  
  -- FAILED  
  cursor_used text,  
  records_read integer default 0,  
  records_inserted integer default 0,  
  records_updated integer default 0,
```



```
records_failed integer default 0,  
error_summary text,  
started_at timestamp with time zone default now(),  
finished_at timestamp with time zone  
);
```

Índices recomendados

```
create index idx_pms_sync_jobs_connection  
on pms_sync_jobs(connection_id);  
create index idx_pms_sync_jobs_started  
on pms_sync_jobs(started_at desc);
```

2. Tabla pms_raw_records

Esta es una de las tablas más importantes.

Aquí se guarda **exactamente lo que devuelve el PMS** antes de transformarlo.

Sirve para:

- debugging
 - auditoría
 - replay de sync
 - análisis de errores
-

SQL

```
create table pms_raw_records (  
  id uuid primary key default gen_random_uuid(),  
  connection_id uuid not null references pms_connections(id),  
  entity_type text not null,  
  external_id text,  
  payload jsonb not null,  
  received_at timestamp with time zone default now(),  
  processed boolean default false,  
  processing_error text  
);
```

índice importante

```
create index idx_pms_raw_records_external  
on pms_raw_records(external_id);
```

3. Tabla pms_sync_cursors

Esto permite hacer **sincronización incremental real**.

Ejemplo:

traer reservas modificadas desde:
2026-03-01T10:00:00

SQL

```
create table pms_sync_cursors (  
  id uuid primary key default gen_random_uuid(),  
  connection_id uuid not null references pms_connections(id),  
  entity_type text not null,  
  last_cursor text,  
  last_sync_at timestamp with time zone,  
  updated_at timestamp with time zone default now()  
);
```

índice

```
create unique index idx_pms_sync_cursor_unique  
on pms_sync_cursors(connection_id, entity_type);
```

4. Flujo real de sincronización

Cuando ejecutas:

Sync RESERVATION

Debacu hace esto:

1. crear pms_sync_job
 2. leer cursor
 3. llamar PMS API
 4. guardar raw_records
 5. mapear datos
 6. insertar / actualizar tablas internas
 7. actualizar cursor
 8. cerrar job
-

5. Flujo visual

```
PMS API  
↓  
pms_raw_records  
↓  
Mapper  
↓  
pms_reservations  
pms_guests  
pms_rooms  
↓  
pms_sync_jobs  
↓  
pms_sync_cursors
```

6. Ejemplo real de raw record

En `pms_raw_records.payload` podrías tener algo así:

```
{
  "reservationId": "ABC123",
  "guestName": "John Doe",
  "arrivalDate": "2026-03-20",
  "departureDate": "2026-03-22",
  "roomType": "DOUBLE",
  "status": "CONFIRMED"
}
```

Esto se transforma en:

`pms_reservations`

7. Ventaja enorme de guardar raw records

Si el PMS cambia el formato de API:

- puedes **reprocesar datos antiguos**
- puedes **debuggear errores**
- puedes **analizar diferencias**

Esto es algo que muchas integraciones no hacen y luego es imposible depurar.

8. Gestión de duplicados

La clave para evitar duplicados es:

```
connection_id
+
external_id
+
entity_type
```

Por ejemplo en `pms_reservations`.

ejemplo

```
create unique index idx_reservation_external
on pms_reservations(connection_id, external_id);
```

9. Control de errores

Si falla un registro:

`records_failed +1`

y guardas el error.

Ejemplo:

```
room_type_external_id not found
```

10. Ejemplo de job real

pms_sync_jobs

campo	valor
entity_type	RESERVATION
sync_mode	INCREMENTAL
records_read	120
inserted	5
updated	115
failed	0

11. limpieza de raw records

Los raw records pueden crecer mucho.

Recomendación:

```
delete from pms_raw_records
where received_at < now() - interval '90 days';
```

Puedes hacerlo con un **cron job semanal**.

12. tabla adicional recomendable

Muchos sistemas añaden:

```
pms_sync_errors
pero en Debacu no es necesario al principio porque:
pms_sync_jobs.error_summary
+
pms_raw_records.processing_error
ya cubren eso.
```

13. resumen de arquitectura PMS Debacu

Tablas principales:

```
pms_connections
pms_sync_jobs
pms_sync_cursors
```

pms_raw_records
pms_room_types
pms_rooms
pms_guests
pms_reservations
pms_stays

14. lo más importante que debes recordar

El conector PMS no es solo API.

Es:

pipeline de datos
+
auditoría
+
replay
+
debugging

15. siguiente paso recomendable

Ahora hay dos piezas importantes que conviene diseñar antes de integrar PMS reales:

- 1 **arquitectura de rate limit + retry automático**
- 2 **arquitectura de webhook PMS**

Esto es crítico porque muchos PMS tienen:

50 requests/min
100 requests/min

Si quieres, en el siguiente paso te preparo **la arquitectura completa de rate limiting y reintentos que usan los conectores SaaS profesionales.**

Perfecto. Vamos con la **arquitectura de rate limiting + reintentos** para los conectores PMS de Debacu.

Esto no es un detalle menor. Si lo ignoras, cuando conectes PMS reales acabarás con:

- bloqueos por exceso de llamadas,
- syncs incompletos,
- jobs que fallan aleatoriamente,
- datos a medias,
- y debugging desagradable.

La idea correcta es que Debacu trate cada PMS como una API externa con restricciones, latencia variable y fallos temporales.

1. Objetivo

Debacu debe poder sincronizar con PMS externos de forma controlada, evitando:

- superar límites de peticiones,
- reintentar de forma agresiva,
- duplicar trabajo innecesariamente,
- corromper cursores,
- lanzar varios syncs incompatibles a la vez.

2. Qué problemas hay que cubrir

2.1 Rate limits del proveedor

Ejemplos típicos:

- 50 req/min
- 100 req/min
- 5 req/seg
- burst pequeño + ventana temporal

2.2 Fallos temporales

Muy comunes:

- 429 Too Many Requests
- 500
- 502
- 503
- 504
- timeouts
- desconexiones

2.3 Fallos permanentes

No conviene reintentar aquí:

- credenciales inválidas
- permisos insuficientes
- endpoint erróneo

- payload inválido
- entidad no soportada

2.4 Concurrency

No debería haber dos syncs a la vez para la misma:

- conexión,
- entidad,
- ventana de sync,

salvo que lo controles muy bien.

3. Principio general

Debe separar tres capas:

1. **política del proveedor**
2. **cliente HTTP robusto**
3. **orquestración del sync**

4. Modelo recomendado

4.1 Política por proveedor

Cada proveedor debería declarar algo así:

```
type ProviderRateLimitPolicy = {
  providerCode: string;
  requestsPerMinute: number;
  maxConcurrentRequests: number;
  retryableStatusCodes: number[];
  nonRetryableStatusCodes: number[];
  maxRetries: number;
  baseDelayMs: number;
  maxDelayMs: number;
  timeoutMs: number;
};
```

Ejemplo Tesipro provisional:

```
const TESIPRO_POLICY: ProviderRateLimitPolicy = {
  providerCode: "TESIPRO_ULYSES",
  requestsPerMinute: 60,
  maxConcurrentRequests: 1,
  retryableStatusCodes: [429, 500, 502, 503, 504],
  nonRetryableStatusCodes: [400, 401, 403, 404, 422],
  maxRetries: 4,
  baseDelayMs: 1000,
  maxDelayMs: 15000,
  timeoutMs: 30000,
};
```

4.2 Cliente HTTP con backoff

Cada adapter no debería usar `fetch` directo sin control. Debe pasar por un wrapper.

4.3 Locks por conexión

Para una misma `connection_id`, mejor ejecutar un solo sync por vez.

5. Estrategia de reintentos correcta

No todo error merece reintento.

Reintentar sí

- 429
- 500
- 502
- 503
- 504
- timeout
- network reset

No reintentar

- 400
- 401
- 403
- 404
- 422

Porque ahí el problema suele ser real y persistente.

6. Backoff recomendado

Usa **exponential backoff con jitter**.

No uses pausas fijas. Eso es mediocre y hace que varios procesos colisionen otra vez al mismo tiempo.

Fórmula razonable

`delay = min(maxDelayMs, baseDelayMs * 2^attempt) + random(0..250ms)`

Ejemplo:

- intento 1 → ~1s
- intento 2 → ~2s
- intento 3 → ~4s
- intento 4 → ~8s

7. Wrapper HTTP recomendado

HttpClientWithRetry.ts

```
// supabase/functions/_shared/pms/http/HttpClientWithRetry.ts
export type RetryPolicy = {
  maxRetries: number;
  baseDelayMs: number;
  maxDelayMs: number;
  timeoutMs: number;
  retryableStatusCodes: number[];
  nonRetryableStatusCodes: number[];
};

function sleep(ms: number) {
  return new Promise((resolve) => setTimeout(resolve, ms));
}

function computeBackoffDelay(
  attempt: number,
  baseDelayMs: number,
  maxDelayMs: number,
): number {
  const exp = Math.min(maxDelayMs, baseDelayMs * 2 ** attempt);
  const jitter = Math.floor(Math.random() * 250);
  return exp + jitter;
}

export class HttpClientWithRetry {
  constructor(private policy: RetryPolicy) {}
  async requestJson<T>(
    input: RequestInfo | URL,
    init?: RequestInit,
  ): Promise<T> {
    let lastError: unknown;
    for (let attempt = 0; attempt <= this.policy.maxRetries; attempt++) {
      const controller = new AbortController();
      const timeoutId = setTimeout(() => {
        controller.abort();
      }, this.policy.timeoutMs);
      try {
        const response = await fetch(input, {
          ...init,
          signal: controller.signal,
        });
        clearTimeout(timeoutId);
        if (response.ok) {
          return (await response.json()) as T;
        }
        if (this.policy.nonRetryableStatusCodes.includes(response.status)) {
          throw new Error(`Non-retryable HTTP error ${response.status}`);
        }
        if (
```

```

        this.policy.retryableStatusCodes.includes(response.status) &&
        attempt < this.policy.maxRetries
    ) {
        const retryAfter = response.headers.get("retry-after");
        const retryAfterMs = retryAfter
            ? Number(retryAfter) * 1000
            : computeBackoffDelay(
                attempt,
                this.policy.baseDelayMs,
                this.policy.maxDelayMs,
            );
        await sleep(retryAfterMs);
        continue;
    }
    throw new Error(`HTTP error ${response.status}`);
} catch (error) {
    clearTimeout(timeoutId);
    lastError = error;
    const isAbort =
        error instanceof DOMException && error.name === "AbortError";
    if (attempt >= this.policy.maxRetries) {
        break;
    }
    if (isAbort || error instanceof TypeError || error instanceof Error) {
        const delay = computeBackoffDelay(
            attempt,
            this.policy.baseDelayMs,
            this.policy.maxDelayMs,
        );
        await sleep(delay);
        continue;
    }
    break;
}
}
throw lastError instanceof Error
    ? lastError
    : new Error("HTTP request failed after retries");
}
}

```

8. Política por proveedor

ProviderPolicies.ts

```

// supabase/functions/_shared/pms/http/ProviderPolicies.ts
import type { RetryPolicy } from "../HttpClientWithRetry.ts";
import type { PmsProviderCode } from "../../types/pms.types.ts";
const DEFAULT_POLICY: RetryPolicy = {
    maxRetries: 4,
    baseDelayMs: 1000,
    maxDelayMs: 15000,
    timeoutMs: 30000,
    retryableStatusCodes: [429, 500, 502, 503, 504],
    nonRetryableStatusCodes: [400, 401, 403, 404, 422],
};
const TESIPRO_POLICY: RetryPolicy = {
    ...DEFAULT_POLICY,
    maxRetries: 4,
    baseDelayMs: 1200,
};

```

```
export function getProviderRetryPolicy(
  providerCode: PmsProviderCode,
): RetryPolicy {
  switch (providerCode) {
    case "TESIPRO_ULYSES":
      return TESIPRO_POLICY;
    default:
      return DEFAULT_POLICY;
  }
}
```

9. Aplicarlo en TesiproConnector

Cuando tengas API real, el connector debería usar el cliente robusto, no fetch a pelo.

Ejemplo de estructura:

```
import { HttpClientWithRetry } from "../http/HttpClientWithRetry.ts";
import { getProviderRetryPolicy } from "../http/ProviderPolicies.ts";
const client = new HttpClientWithRetry(
  getProviderRetryPolicy("TESIPRO_ULYSES"),
);
```

Y luego:

```
const data = await client.requestJson<unknown>(url, {
  method: "GET",
  headers: {
    Authorization: `Bearer ${token}`,
    "Content-Type": "application/json",
  },
});
```

10. Locks de sincronización

Esto es importante. Debes evitar dos sync simultáneos de la misma conexión y entidad.

Regla recomendable

No permitir dos jobs RUNNING para:

- mismo connection_id
- mismo entity_type

Enfoque simple

Antes de crear un job nuevo, comprobar si ya existe uno RUNNING.

SQL útil

```
create index if not exists idx_pms_sync_jobs_running
on pms_sync_jobs(connection_id, entity_type, status);
```

Validación lógica

Antes de crear job:

```
select id
from pms_sync_jobs
where connection_id = ?
      and entity_type = ?
      and status = 'RUNNING'
limit 1
```

Si existe, abortas con error controlado:

`sync_already_running`

11. Cursor update seguro

No actualices `pms_sync_cursors` hasta que el job haya terminado bien.

Correcto

1. lees cursor actual
2. haces fetch
3. persistes raw
4. persistes canonical
5. cierras job como SUCCESS
6. actualizas cursor

Incorrecto

Actualizar cursor antes de guardar datos.

Si el proceso falla entre medias, pierdes eventos.

12. Sync parcial

A veces un sync no falla del todo. Puede traer 100 registros y fallar 3.

Ahí tienes dos opciones:

conservadora

si falla alguno, job FAILED

más útil

si mayoría entra y algunos fallan, job PARTIAL

Para Debaqu, yo haría:

- SUCCESS si `records_failed = 0`
- PARTIAL si `records_failed > 0` y hubo inserciones/updates
- FAILED si no se pudo procesar nada o falló antes de ingestión real

13. Rate limiting práctico

Con Edge Functions no montaría aún un token bucket complejo distribuido. Sería sobrediseño prematuro.

Haría esto:

fase actual

- `maxConcurrentRequests` = 1 por conexión
- procesamiento secuencial
- reintentos con backoff
- pausas entre páginas si hace falta

más adelante

Si un PMS exige mucha paginación o tienes gran volumen:

- cola por conexión
- scheduler
- throttle persistido en DB o Redis-like layer

Ahora no hace falta llegar tan lejos.

14. Pausa entre páginas

Si un endpoint paginado devuelve cientos de páginas, conviene meter pausa pequeña entre páginas.

Ejemplo:

- 200ms
- 300ms

No por necesidad técnica pura, sino por prudencia.

```
await sleep(250);
```

Eso reduce picos y hace el comportamiento más civilizado.

15. Qué guardar en `pms_sync_jobs`

Añadiría si no lo tienes todavía:

- `attempt_count`
- `cursor_used`
- `next_cursor`

- `rate_limited_count`

Campos recomendables:

```
alter table pms_sync_jobs
add column if not exists attempt_count integer not null default 0;
alter table pms_sync_jobs
add column if not exists next_cursor text null;
alter table pms_sync_jobs
add column if not exists rate_limited_count integer not null default 0;
```

16. Qué guardar cuando hay 429

Cuando recibas 429:

- incrementa `rate_limited_count`
- respeta `Retry-After` si viene
- deja trazabilidad en logs

No hace falta aún una tabla especial para eso.

17. Error classification

Conviene clasificar errores, no solo guardar texto libre.

Añadiría a `pms_sync_jobs`:

```
alter table pms_sync_jobs
add column if not exists error_code text null;
```

Ejemplos:

- `rate_limited`
- `auth_failed`
- `provider_unavailable`
- `network_timeout`
- `invalid_payload`
- `sync_already_running`

Eso mejora muchísimo el panel admin.

18. Servicio de lock simple

`PmsSyncGuardService.ts`

```
// supabase/functions/_shared/pms/services/PmsSyncGuardService.ts
import { getServiceClient } from "../../db.ts";
export class PmsSyncGuardService {
  private db = getServiceClient();
```

```

async assertNoRunningJob(connectionId: string, entityType: string) {
  const { data, error } = await this.db
    .from("pms_sync_jobs")
    .select("id")
    .eq("connection_id", connectionId)
    .eq("entity_type", entityType)
    .eq("status", "RUNNING")
    .limit(1);
  if (error) throw error;
  if ((data ?? []).length > 0) {
    throw new Error("sync_already_running");
  }
}
}

```

Y lo llamas al principio de `PmsSyncService.run`.

19. Diseño realista para Debacu ahora

Yo haría ya, sin esperar más:

1. wrapper HTTP con retry
2. política por proveedor
3. guard contra sync concurrente
4. cursor update solo al final
5. campos extra en `pms_sync_jobs`
6. panel admin mostrando `error_code` y `rate_limited_count`

Eso ya es serio.

20. Lo que no haría aún

No haría todavía:

- cola distribuida compleja
- worker pool multi-tenant sofisticado
- scheduler muy avanzado
- prioridad dinámica por PMS

Eso es para cuando tengas varios PMS reales y volumen. Ahora sería meter complejidad antes de tener la necesidad.

21. Siguiendo paso correcto

Después de esto, la siguiente pieza importante es **arquitectura de webhooks PMS**.

Porque en cuanto un PMS te permita empujar eventos, no conviene seguir dependiendo solo de polling.

Ahí hay que diseñar:

- endpoint webhook
- validación de firma
- raw event storage
- idempotencia
- replay
- encolado a sync

Eso sí merece hacerse bien antes de producción.

Perfecto. Vamos con la **arquitectura de webhooks PMS para Debacu**. Aunque al principio algunos PMS solo permitan **polling**, diseñarlo bien desde el inicio te evita rehacer medio sistema cuando conectes PMS más modernos (Mews, Apaleo, Cloudbeds, etc.).

El objetivo es que Debacu pueda **recibir eventos del PMS** y procesarlos de forma segura, idempotente y auditable.

1. Qué son los webhooks en PMS

Un webhook es un **evento HTTP enviado por el PMS a Debacu** cuando ocurre algo.

Ejemplos típicos:

- nueva reserva creada
- reserva modificada
- check-in
- check-out
- huésped actualizado
- cancelación

En lugar de que Debacu pregunte cada X minutos (polling), el PMS avisa.

Ejemplo:

```
POST https://api.debacu.com/pms/webhook
{
  "event": "reservation.updated",
  "reservationId": "ABC123",
  "timestamp": "2026-03-16T10:23:11Z"
}
```

2. Principio fundamental

Nunca proceses un webhook directamente.

El flujo correcto es:

```
Webhook HTTP
↓
pms_webhook_events (guardar raw)
↓
event processor
↓
sync específico
↓
tablas Debacu
```

Es decir, **primero guardar, luego procesar**.

3. Tabla pms_webhook_events

Aquí se guarda **cada evento recibido**.

SQL

```
create table pms_webhook_events (
  id uuid primary key default gen_random_uuid(),
  provider_code text not null,
  connection_id uuid references pms_connections(id),
  event_type text,
  external_event_id text,
  payload jsonb not null,
  received_at timestamp with time zone default now(),
  processed boolean default false,
  processing_error text,
  processed_at timestamp with time zone
);
```

índice importante

```
create index idx_webhook_provider
on pms_webhook_events(provider_code);
create index idx_webhook_processed
on pms_webhook_events(processed);
```

4. Idempotencia

Muchos PMS **reenvían webhooks si no reciben 200 OK**.

Por eso debes evitar duplicados.

Clave típica:

```
provider_code
+
external_event_id
```

índice único

```
create unique index idx_webhook_unique
on pms_webhook_events(provider_code, external_event_id);
```

Si el PMS no envía event_id, puedes usar hash del payload.

5. Endpoint webhook Debacu

Edge Function recomendada:

supabase/functions/pms_webhook_receiver

pms_webhook_receiver/index.ts

Ejemplo simplificado:

```
import "jsr:@supabase/functions-js/edge-runtime.d.ts";
import { json, preflight } from "../shared/cors.ts";
import { supabaseServiceClient } from "../shared/auth.ts";
Deno.serve(async (req) => {
  if (req.method === "OPTIONS") return preflight(req);
  try {
    const body = await req.json();
    const db = supabaseServiceClient();
    const { error } = await db
      .from("pms_webhook_events")
      .insert({
        provider_code: "TESIPRO_ULYSES",
        payload: body,
        event_type: body.event ?? null,
        external_event_id: body.eventId ?? null
      });
    if (error) throw error;
    return json({ ok: true });
  } catch (error) {
    return json({
      ok: false,
      error: error instanceof Error ? error.message : "unknown_error"
    }, { status: 500 });
  }
});
```

6. Validación de firma (muy importante)

Muchos PMS envían un **header de firma**.

Ejemplo:

```
X-Signature
X-Hub-Signature
```

X-Mews-Signature

Debacu debe verificar que el webhook viene del PMS real.

Proceso típico:

payload

+

secret

↓

HMAC SHA256

↓

comparar con header

Si no coincide → rechazar webhook.

Esto evita:

- spoofing
 - ataques
 - eventos falsos
-

7. Worker de procesamiento

Después de guardar el webhook, un worker lo procesa.

Puedes hacerlo con:

- cron job
 - job scheduler
 - función edge periódica
-

flujo

```
pms_webhook_events
  where processed=false
  ↓
processor
  ↓
detectar tipo evento
  ↓
llamar sync específico
  ↓
marcar processed=true
```

8. Ejemplo de processor

Pseudo lógica:

```
for each event where processed=false
  if event_type == reservation.created
```

```
run reservation sync
if event_type == reservation.updated
  run reservation sync
if event_type == checkin
  run stay sync
mark processed
```

9. Estrategia recomendada para Debacu

No intentes mapear todos los campos del webhook.

Los webhooks suelen tener **poca información**.

Lo correcto es:

webhook → trigger → sync incremental

Ejemplo:

```
reservation.updated
```

↓

```
sync reservation ABC123
```

No intentes actualizar directamente tablas Debacu desde el webhook.

10. Ventaja enorme

Con este modelo puedes tener:

polling

+

webhooks

al mismo tiempo.

Muchos sistemas SaaS lo hacen así.

11. Manejo de errores

Si falla el procesamiento:

```
processed = false
```

```
processing_error = error
```

Luego puedes reintentar.

12. retry automático

Ejemplo de worker:

```
si error
```

```
  retry 3 veces
```

```
si sigue fallando
```

13. reprocessing manual

Muy útil tener función:

`reprocess_webhook_event(event_id)`

Esto permite debug.

14. Panel admin Debacu

La pantalla PMS Admin debería tener:

Tabla:

webhook events

Columnas:

- provider
- event_type
- received_at
- processed
- error

Acciones:

Reprocess

View payload

15. ejemplo de flujo completo

PMS

↓

Webhook

↓

pms_webhook_events

↓

processor

↓

pms_sync_run

↓

Debacu tables

16. qué PMS soportan webhooks

PMS	Webhooks
Mews	sí
Apaleo	sí
Cloudbeds	sí
Opera Cloud	sí
Tesipro	depende

Por eso hay que soportar ambos modelos.

17. ventaja estratégica para Debacu

Con polling solo:

latencia 5-15 min

Con webhooks:

latencia segundos

Esto mejora mucho:

- detección de fraude
 - evaluación de riesgo
 - análisis revenue
-

18. seguridad recomendada

Añadir campos a pms_connections:

webhook_secret
webhook_url

19. arquitectura completa PMS Debacu

Finalmente queda:

pms_connections
pms_sync_jobs
pms_sync_cursors
pms_raw_records
pms_webhook_events
pms_rooms
pms_room_types
pms_guests
pms_reservations
pms_stays

20. consejo realista

Aunque diseñes webhooks ahora, **no dependas de ellos al principio**.

Primero asegúrate de que el **polling funciona perfecto**.

Luego activas webhooks cuando el PMS lo permita.

21. siguiente paso importante

El siguiente paso clave para Debacu sería diseñar **el modelo de permisos PMS por organización y propiedad**.

Porque cuando conectes PMS reales aparecerá este problema:

un PMS
varios hoteles
varios usuarios
distintas propiedades

Si quieres, en el siguiente paso te preparo **la arquitectura multi-hotel PMS para Debacu (muy importante para no romper el modelo de organizaciones)**.

perfecto. Vamos con la **arquitectura multi-hotel / multi-propiedad para integraciones PMS en Debacu**. Esto es clave porque tu SaaS ya funciona con **organizaciones que tienen varias propiedades**, y el PMS también puede trabajar así. Si el modelo no se diseña bien ahora, después aparecen problemas de permisos, datos mezclados entre hoteles o sincronizaciones incorrectas.

Voy directo a lo práctico.

1. Modelo conceptual Debacu

En Debacu hay tres niveles:

Organización

↓

Propiedades (hoteles)

↓

Datos PMS (reservas, huéspedes, habitaciones...)

Ejemplo real:

Organización: Grupo Hotelero Costa

Propiedades:

Hotel Costa Norte

Hotel Costa Centro

Hotel Costa Playa

Todos usan **el mismo PMS (Ulyses)**.

2. Regla de oro

La conexión PMS debe pertenecer siempre a una propiedad, no solo a la organización.

Esto evita mezclar datos.

3. Tabla pms_connections

Cada conexión representa una integración entre una propiedad y un PMS.

```
create table pms_connections (  
  id uuid primary key default gen_random_uuid(),  
  org_id uuid not null,  
  property_id uuid not null,  
  provider_code text not null,  
  environment text not null default 'sandbox',  
  auth_mode text not null,  
  credentials_json jsonb,  
  status text not null default 'PENDING',  
  last_sync_at timestamp with time zone,  
  last_success_sync_at timestamp with time zone,  
  last_error_at timestamp with time zone,  
  last_error_message text,  
  webhook_secret text,  
  webhook_url text,  
  created_at timestamp with time zone default now(),  
  updated_at timestamp with time zone default now()  
);
```

4. Relación con propiedades Debacu

Tu sistema ya tiene algo parecido a:

debacu_eval_organizations
debacu_eval_properties

Entonces la relación correcta es:

```
organizations  
  ↓  
properties  
  ↓  
pms_connections
```

5. Ejemplo real

org	property	PMS
Grupo Costa	Costa Norte	TESIPRO
Grupo Costa	Costa Centro	TESIPRO
Grupo Costa	Costa Playa	TESIPRO

Cada uno tiene su propia conexión.

6. Por qué no conectar PMS a nivel organización

Muchos PMS tienen datos mezclados de varios hoteles.

Si conectas Debacu así:

org → PMS

puede pasar que:

- reservas del hotel A aparezcan en hotel B
- habitaciones incorrectas
- revenue mezclado

Por eso **la conexión debe ser por propiedad**.

7. Cómo mapear propiedades PMS

Muchos PMS usan su propio ID de propiedad.

Ejemplo:

Ulyses property_id = 342

Debacu property_id = uuid

Necesitas tabla de mapping.

pms_property_mappings

```
create table pms_property_mappings (  
  id uuid primary key default gen_random_uuid(),  
  connection_id uuid not null references pms_connections(id),  
  external_property_id text not null,  
  debacu_property_id uuid not null,  
  created_at timestamp with time zone default now()  
);
```

8. Cómo funciona en runtime

Cuando llega un registro del PMS:

```
{  
  "propertyId": "342",  
  "reservationId": "ABC123"  
}
```

Debacu hace:

9. Aislamiento de datos

Todas las tablas PMS deben tener:

connection_id
property_id
org_id

Ejemplo pms_reservations:

```
create table pms_reservations (  
  id uuid primary key default gen_random_uuid(),  
  connection_id uuid not null,  
  org_id uuid not null,  
  property_id uuid not null,  
  external_id text not null,  
  arrival_date date,  
  departure_date date,  
  status text,  
  created_at timestamp with time zone default now()  
);
```

10. Ventaja de este modelo

Esto te permite:

1 PMS
10 hoteles
100 usuarios
sin mezclar datos.

11. Control de permisos

Debacu debe permitir:

usuarios organización

ven todos los hoteles

usuarios hotel

ven solo su propiedad

ejemplo

usuario	acceso
admin grupo	todos

usuario	acceso
manager hotel	solo su hotel

12. Filtro obligatorio en consultas

Toda consulta PMS debe incluir:

```
org_id
+
property_id
```

Ejemplo:

```
select *
from pms_reservations
where org_id = ?
and property_id = ?
```

13. Integración con selector global Debacu

Tú ya tienes un **selector de propiedad en la app**.

Ese selector debe determinar:

```
connection_id
```

para el PMS.

14. Ejemplo de flujo real

Usuario entra en Debacu.

Selecciona:

Hotel Costa Norte

Debacu busca:

```
pms_connections
where property_id = costa_norte
```

y usa esa conexión.

15. Caso especial: PMS multi-hotel

Algunos PMS (Opera, Protel, etc.) tienen **una API para todos los hoteles**.

Entonces:

```
1 conexión
+
mappings
```

Pero internamente Debacu sigue separando.

16. Arquitectura final

```
organizations
  ↓
properties
  ↓
pms_connections
  ↓
pms_sync_jobs
pms_raw_records
pms_sync_cursors
pms_webhook_events
  ↓
pms_room_types
pms_rooms
pms_guests
pms_reservations
pms_stays
```

17. Algo muy importante

Nunca confíes en que el PMS filtre por hotel.

Siempre filtra tú también.

18. Estrategia de onboarding

Cuando conectes un PMS:

```
1 crear conexión
2 probar credenciales
3 obtener lista propiedades PMS
4 crear mapping
5 iniciar sync
```

19. Ejemplo onboarding Tesipro

```
connect Ulyses
  ↓
listar hoteles PMS
  ↓
mapear con Debacu properties
  ↓
sync inicial
```

20. ventaja estratégica para Debacu

Con este modelo podrás soportar:

- hoteles independientes
- cadenas hoteleras
- multi-PMS
- migraciones PMS

sin reestructurar la base.

21. siguiente paso importante

Ahora mismo tu arquitectura PMS ya tiene:

- sync pipeline
- retry system
- webhooks
- multi-property

El siguiente paso realmente importante para Debacu sería diseñar **el modelo de Revenue Data ingestion desde PMS** (ADR, RevPAR, folios, cargos, etc.).

Ese módulo es el que conecta **Debacu Evaluation + Debacu Revenue Intelligence**.

Si quieres, en el siguiente paso te preparo **la arquitectura profesional de ingestión revenue desde PMS (la que usan los RMS reales)**.

A partir de aquí falta:

Perfecto. Te dejo un **listado claro y ordenado de lo que falta** para que Debacu pase de “arquitectura preparada” a **conector PMS funcionando con datos reales**. Está priorizado para que mañana sepamos exactamente por dónde continuar.

1. Obtener acceso API del PMS (Tesipro / Ulyses)

Esto es lo primero que bloquea todo lo demás.

Necesitas pedir a Tesipro:

- documentación API

- tipo de API (REST / SOAP)
- endpoint base
- método de autenticación
- scopes o permisos
- sandbox o entorno test
- límites de rate
- endpoints disponibles

Endpoints mínimos necesarios:

reservations
guests
rooms
room types
stays / checkin checkout
Opcional para revenue:

folios
charges

2. Confirmar modelo de autenticación

Debes saber cuál de estos usa Ulyses:

API Key
OAuth2
Basic Auth
Token session
y dónde se envía:

header
query
body

3. Crear TesiproAdapter real

Implementar llamadas a API real:

fetchRoomTypes()
fetchRooms()
fetchGuests()
fetchReservations()
fetchStays()

Este adapter debe usar:

HttpClientWithRetry
ProviderPolicies

4. Implementar mappers PMS → modelo Debacu

Convertir payload del PMS a modelo Debacu.

Archivos:

```
RoomTypeMapper.ts
RoomMapper.ts
GuestMapper.ts
ReservationMapper.ts
StayMapper.ts
```

Ejemplo:

```
PMS reservationId → external_id
arrivalDate → check_in_date
departureDate → check_out_date
```

5. Implementar sync completo

Integrar el adapter con:

```
PmsSyncService
pms_sync_run
```

Pipeline:

```
API PMS
↓
pms_raw_records
↓
mapper
↓
pms_reservations
pms_guests
pms_rooms
pms_room_types
```

6. Probar primer sync real

Ejecutar:

```
ROOM_TYPE
ROOM
GUEST
RESERVATION
STAY
```

y verificar tablas:

```
pms_room_types
pms_rooms
pms_guests
pms_reservations
pms_stays
```

7. Implementar mapping de propiedades PMS

Si el PMS devuelve su propio `property_id`.

Tabla:

`pms_property_mappings`

Mapeo:

`external_property_id` → `debacu_property_id`

8. Crear función de test de conexión

Edge Function:

`pms_connection_test`

Debe:

`validar credenciales`

`probar endpoint`

`medir latencia`

`verificar permisos`

Devuelve:

`connection_valid`

`api_latency`

`permissions_ok`

9. Ajustar tabla `pms_sync_jobs`

Añadir campos útiles si aún no existen:

`attempt_count`

`next_cursor`

`rate_limited_count`

`error_code`

10. Implementar cron de sincronización

Ejemplo cada:

`5 minutos` → `reservas`

`30 minutos` → `huéspedes`

`1 hora` → `habitaciones`

usando:

`pms_sync_cursors`

11. Preparar limpieza de raw records

Evitar crecimiento infinito:

```
delete raw records > 90 días  
cron semanal.
```

12. Activar panel admin PMS

Pantalla interna con:

```
listar conexiones  
reset conexión  
sync manual  
ver jobs  
ver errores
```

13. Preparar webhooks (si PMS lo soporta)

Endpoint:

```
pms_webhook_receiver  
tabla:
```

```
pms_webhook_events  
flujo:
```

```
webhook → raw event → processor → sync
```

14. Verificar aislamiento multi-propiedad

Confirmar que todas las tablas tienen:

```
org_id  
property_id  
connection_id  
y filtros obligatorios.
```

15. Primer dataset real para Debacu

Cuando el conector funcione debes poder obtener:

```
habitaciones  
tipos de habitación  
huéspedes  
reservas  
estancias
```

Con eso ya podrás empezar:

risk analysis
screening
revenue ingestion

16. Siguiente fase después de conectar PMS

Cuando el primer PMS funcione:

- 1 revenue ingestion
 - 2 ADR / RevPAR pipeline
 - 3 pickup analysis
 - 4 revenue leak detection
-

17. Estado real del proyecto

Área	Estado
Arquitectura PMS	✓
Tablas	✓
Sync pipeline	✓
Retry system	✓
Webhooks	✓
Panel admin	✓
Adapter real PMS	✗
Credenciales PMS	✗
Primer sync real	✗

18. Mañana lo más importante

El primer paso será:

analizar la API real de Tesipro Ulyses

porque dependiendo de cómo sea:

REST
SOAP
Channel manager API
el adapter cambia bastante.

✓ Cuando volvamos mañana dime algo como:

“vemos API Tesipro”

y analizamos exactamente **cómo conectarla a Debacu.**

Para calude: